

SENG 31242

System Design Specification (SDS)

GarageGo

Smart Garage & Emergency Vehicle Repair Platform

By

P. Kasturi – SE/2022/053
K. Kajaluxmy – SE/2022/056
V. Vanushan – SE/2022/031
S. Krishnapiriyani – SE/2022/048

A report submitted in partial fulfilment of the requirements for the degree of
Bachelor of Science Honours in Software Engineering (B.Sc.SE)

Software Engineering Teaching Unit
Faculty of Science
University of Kelaniya
Sri Lanka

2026

Abstract

This System Design Specification (SDS) document describes the complete technical design of the GarageGo platform, a multi-sided mobile and web application for vehicle repair, scheduled garage booking, and emergency roadside assistance in Sri Lanka. The document is derived directly from the approved GarageGo Software Requirements Specification (SRS v0.1) and covers the high-level architectural design, Architectural Decision Records (ADRs), database schema with Entity-Relationship diagram, UML class diagram, sequence diagrams for all ten use cases, state machine diagrams for stateful entities, user interface wireframes and navigation flow, security design, external and internal interface specifications, and non-functional design decisions addressing all fourteen NFRs defined in the SRS.

The system is designed using a Layered plus Event-Driven hybrid architecture with a Node.js/Express REST API backend, Socket.io for real-time bidirectional communication, React Native (Expo) for cross-platform mobile applications, React.js for the Admin Web Dashboard, and PostgreSQL 16 as the primary relational data store with Redis 7 as the caching and session layer. All components are containerised using Docker and deployable behind a load balancer for horizontal scaling. External integrations include Google Maps Platform for geocoding, routing, and GPS tracking, Twilio for SMS OTP verification, and Firebase Cloud Messaging for push notifications.

Keywords- System Design, Layered Architecture, Event-Driven Architecture, Node.js, React Native, PostgreSQL, Redis, Socket.io, JWT Authentication, Docker, Google Maps, Emergency SOS Dispatch, UML, ER Diagram, OWASP.

Definitions, Acronyms, and Abbreviations

Table 1: Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition / Description
SDS	System Design Specification
SRS	Software Requirements Specification
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token – stateless authentication token
OTP	One-Time Password – time-limited SMS verification code
GPS	Global Positioning System – satellite-based location tracking
SOS	Emergency roadside assistance request triggered by a customer breakdown
ETA	Estimated Time of Arrival – projected mechanic arrival time
FCM	Firebase Cloud Messaging – push notification service
Socket.io	Real-time bidirectional event-based communication library
Redis	In-memory data structure store for caching, sessions, and queues
MVC	Model-View-Controller architectural pattern
ADR	Architectural Decision Record
ER	Entity-Relationship diagram
UML	Unified Modelling Language
RBAC	Role-Based Access Control
TLS	Transport Layer Security
OWASP	Open Web Application Security Project
CI/CD	Continuous Integration / Continuous Deployment
NFR	Non-Functional Requirement
FR	Functional Requirement
UC	Use Case
2W	Two-wheeler vehicle (motorcycle, scooter, three-wheeler)
4W	Four-wheeler vehicle (car, van, SUV, jeep)
NIC	National Identity Card – Sri Lankan government-issued identification
MVP	Minimum Viable Product
DXA	Document Exchange Absolute unit (used in Word formatting)

Chapter 1: Architectural Design

This chapter presents the high-level system architecture of GarageGo, justification for the chosen design pattern using Architectural Decision Records, component and deployment diagrams, and the technology stack with rationale. All design decisions are directly traceable to the functional and non-functional requirements documented in the approved GarageGo SRS v0.1.

1.1 High-Level Architecture

GarageGo adopts a Layered + Event-Driven hybrid architecture. The layered component provides clear separation of concerns between the presentation, API, business logic, data access, and persistence tiers. The event-driven component, implemented via Socket.io over WebSocket, handles all real-time interactions: emergency SOS broadcasting, first-accept-wins assignment, and mechanic GPS tracking. This hybrid approach was selected because GarageGo's core value proposition — real-time emergency dispatch and live GPS tracking — cannot be delivered by a purely request-response REST architecture alone, while a fully event-driven architecture would complicate the structured booking and administrative workflows that are well-served by REST.

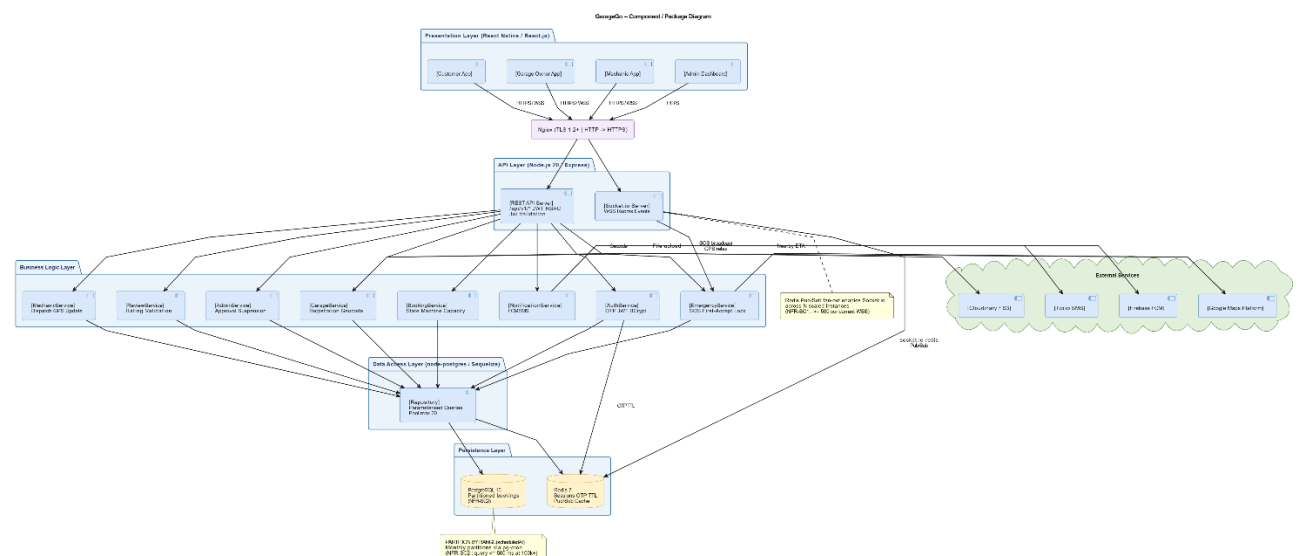


Figure 1: GarageGo – High-Level System Architecture Diagram [see documents/diagrams/component_diagram.png]

Table 2: Architectural Layers and Responsibilities

Layer	Technology / Component	Responsibility
Presentation	React Native (Expo) – Customer, Garage, Mechanic Apps; React.js – Admin Dashboard	Renders mobile and web UIs; handles user interactions; communicates with the API over HTTPS and WSS.
API Gateway	Node.js/Express, JWT Middleware, TLS 1.2+	Authenticates JWT tokens; enforces RBAC; validates request schema; routes to service handlers.
Real-Time Event Bus	Socket.io (WebSocket over WSS)	Broadcasts emergency SOS alerts; streams mechanic GPS updates every 5 seconds; delivers booking status events.
Business Logic	Express Route Handlers, EmergencyService, BookingService, GarageService, NotificationService	Implements all business rules; orchestrates SELECT FOR UPDATE database locks; manages booking state machine transitions.
Cache / Session	Redis 7	Stores active Socket.io sessions; caches garage lists; manages OTP expiry TTLs; queues reconciliation jobs.
Data Access	node-postgres (pg), Sequelize ORM	Abstracts PostgreSQL queries; manages connection pooling; enforces parameterised queries.
Database	PostgreSQL 15 (partitioned bookings table)	Persists all system data; provides ACID transactions; supports monthly horizontal partitioning per NFR-SC2.
External Services	Google Maps API, Twilio (SMS OTP), Firebase Cloud Messaging	Geocoding, routing, and distance calculation; phone number verification; push notification delivery.

1.2 Architectural Decision Records

All major design decisions are documented as Architectural Decision Records (ADRs) per Section 7.2 of the course guidelines. Each ADR follows the template: Context, Decision, Rationale, Alternatives Considered, and Consequences.

ADR-01: Architectural Pattern Selection

Field	Content
Date	2026-06-12
Status	Accepted
Context	GarageGo requires both structured request-response workflows (booking creation, garage registration, admin approval) and real-time bidirectional communication

Field	Content
	(emergency SOS broadcast, GPS tracking). A single architectural pattern must support both without excessive complexity for a 4-person academic team.
Decision	Adopt a Layered + Event-Driven hybrid architecture. REST over HTTPS for all CRUD and transactional endpoints; Socket.io over WSS for real-time event delivery.
Rationale	REST satisfies NFR-P1 (API response $\leq 2s$ under 50 concurrent users) with stateless horizontal scaling per NFR-SC3. Socket.io satisfies NFR-P2 (emergency broadcast $\leq 2s$) and NFR-P3 (GPS relay $\leq 1s$). A microservices pattern was rejected as it introduces deployment and orchestration overhead disproportionate to team size and MVP scope.
Alternatives Considered	Microservices: better isolation but high Kubernetes overhead. Pure REST polling: simpler but cannot meet NFR-P2/P3 real-time thresholds. Full event-driven (Kafka): over-engineered for MVP scale.
Consequences	Developers must manage two server processes (Express + Socket.io); Redis is required for session sharing across scaled instances (NFR-SC1).

ADR-02: Frontend Framework – React Native vs Native

Field	Content
Date	2026-06-12
Status	Accepted
Context	Three mobile apps (Customer, Garage Owner, Mechanic) must run on both iOS 15+ and Android 10+. The team of four has JavaScript/React experience but limited Swift/Kotlin expertise.
Decision	React Native with Expo managed workflow for all three mobile apps.
Rationale	Single shared codebase reduces development effort by approximately 40% compared to maintaining separate iOS and Android codebases. Expo provides pre-built modules for GPS location (required for NFR-P3), push notifications (FR-14), and camera (garage document upload in FR-10). The design constraint in SRS Section 2.4 explicitly mandates cross-platform development.
Alternatives Considered	Native iOS + Android: best performance but doubles development cost. Flutter: strong cross-platform but team has no Dart experience. PWA: insufficient access to GPS always-on and background location required by FR-07.
Consequences	Native module access is limited to Expo SDK; custom native modules require ejecting. React Native bridge overhead is acceptable for MVP but may require optimization at scale.

ADR-03: Database Selection – PostgreSQL vs NoSQL

Field	Content
Date	2026-06-12
Status	Accepted
Context	GarageGo stores highly relational data: customers have vehicles, vehicles have bookings, bookings have invoices, garages have mechanics, emergency requests link to garages. The booking state machine (FR-09) requires ACID transaction guarantees

Field	Content
	to prevent double-booking. NFR-SC2 requires monthly horizontal partitioning of the bookings table at 100,000+ records.
Decision	PostgreSQL 15 as the primary relational database with monthly table partitioning on the bookings table.
Rationale	PostgreSQL provides ACID transactions required for SELECT FOR UPDATE locks in first-accept-wins assignment (FR-06). Native declarative partitioning satisfies NFR-SC2 without additional infrastructure. Foreign key constraints enforce referential integrity across the relational domain model. The SRS Section 6.2.1 explicitly lists PostgreSQL as the chosen database.
Alternatives Considered	MongoDB: flexible schema but weak transaction support and no native JOIN for relational queries. MySQL: viable alternative but lacks advisory locks used in emergency assignment. Firebase Firestore: real-time sync is useful but insufficient for complex relational queries and ACID requirements.
Consequences	Schema migrations require careful planning; partition management must be automated by a scheduled job. PostgreSQL Multi-AZ replication is deferred to post-MVP.

1.3 Scalability and Maintainability Considerations

- Horizontal scaling is achieved by deploying the Node.js/Express API as multiple Docker containers behind a load balancer; Redis stores Socket.io session state shared across all instances.
- The stateless REST API design ensures any backend instance can serve any request without sticky sessions, enabling true horizontal scalability.
- Monthly horizontal partitioning of the PostgreSQL bookings table keeps current-month queries within 500ms at 100,000+ records as required.
- Each GarageGo component (API server, Socket.io server, Admin Dashboard) is independently deployable as a Docker container, enabling redeployment within 5 minutes.

Chapter 2: Database Design

This chapter presents the Entity-Relationship (ER) diagram, entity descriptions with full attribute tables, key relationships, and normalization analysis for the GarageGo database schema. The schema is implemented in PostgreSQL 15. All primary keys use UUID type to ensure globally unique, non-sequential identifiers.

2.1 Entity-Relationship (ER) Diagram

The ER diagram uses standard notation: rectangles for entities, ellipses for attributes, underlined ellipses for primary keys, and diamonds for relationships. Cardinality labels (1 and N/M) are placed on connecting lines.

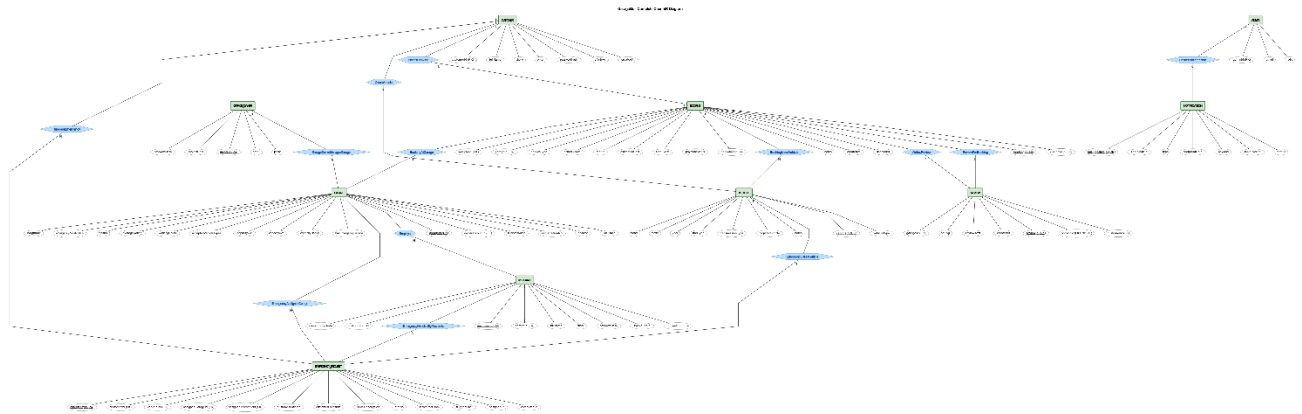


Figure 2: GarageGo – Entity-Relationship (ER) Diagram [see documents/diagrams/ER_diagram.png]

2.2 Entity Descriptions and Attributes

Customer

The Customer entity stores registered vehicle owners using the GarageGo platform.

Table 3: Customer Entity Attributes

Attribute	Data Type	Constraint	Description
customerId (PK)	UUID	NOT NULL, UNIQUE	System-generated primary key
fullName	VARCHAR(150)	NOT NULL	Customer full name
phone	VARCHAR(20)	NOT NULL, UNIQUE	Verified via OTP (E.164 format)
email	VARCHAR(255)	NOT NULL, UNIQUE	Login email
passwordHash	VARCHAR(255)	NOT NULL	BCrypt hashed password
isActive	BOOLEAN	DEFAULT TRUE	Account status
createdAt	TIMESTAMP	NOT NULL	Account creation time

Vehicle

The Vehicle entity records vehicles registered by customers for use in bookings and emergency SOS requests.

Table 4: Vehicle Entity Attributes

Attribute	Data Type	Constraint	Description
vehicleId (PK)	UUID	NOT NULL, UNIQUE	System-generated surrogate primary key
customerId (FK)	UUID	NOT NULL	References Customer.customerId
vehicleType	ENUM	NOT NULL	TWO_WHEEL FOUR_WHEEL HEAVY – determines garage capacity filtering
brand	VARCHAR(100)	NOT NULL	Vehicle manufacturer (e.g., Toyota, Honda)
model	VARCHAR(100)	NOT NULL	Vehicle model name (e.g., Corolla, Dio)
year	SMALLINT	NOT NULL	Manufacturing year (1990–2026)
fuelType	ENUM	NOT NULL	PETROL DIESEL ELECTRIC HYBRID
transmissionType	ENUM	NOT NULL	MANUAL AUTOMATIC
registrationNo	VARCHAR(30)	NOT NULL, UNIQUE	Official Sri Lanka vehicle registration plate (FR-02)
notes	TEXT	NULLABLE	Optional customer notes about the vehicle

Garage

The Garage entity stores registered and approved garages visible to customers for booking and emergency SOS dispatch.

Table 5: Garage Entity Attributes

Attribute	Data Type	Constraint	Description
garageId (PK)	UUID	NOT NULL, UNIQUE	System-generated surrogate primary key
ownerAccountId (FK)	UUID	NOT NULL	References GarageOwner.ownerId
businessName	VARCHAR(200)	NOT NULL	Registered trade name of the garage
registrationNo	VARCHAR(50)	NOT NULL, UNIQUE	Business registration number (FR-10)
address	VARCHAR(300)	NOT NULL	Physical street address
latitude	DECIMAL(9,6)	NOT NULL	WGS84 latitude for Google Maps geocoding
longitude	DECIMAL(9,6)	NOT NULL	WGS84 longitude for Google Maps geocoding

Attribute	Data Type	Constraint	Description
emergencyZoneRadius	INTEGER	NOT NULL	Radius in km within which SOS alerts are broadcast (FR-05)
status	ENUM	NOT NULL	PENDING APPROVED SUSPENDED – approval lifecycle (FR-11)
averageRating	DECIMAL(3,2)	DEFAULT 0.00	Computed average of all Review ratings (FR-13)
workingHours	JSONB	NOT NULL	Operating hours per day as JSON object
acceptedVehicleTypes	TEXT[]	NOT NULL	Array of supported vehicle categories
capacity2W	SMALLINT	NOT NULL, >= 0	Maximum simultaneous two-wheeler bays (FR-08)
capacity4W	SMALLINT	NOT NULL, >= 0	Maximum simultaneous four-wheeler bays (FR-08)
capacityHeavy	SMALLINT	NOT NULL, >= 0	Maximum simultaneous heavy vehicle bays (FR-08)
hasEmergencyService	BOOLEAN	DEFAULT FALSE	TRUE = garage receives SOS broadcasts (FR-05)

GarageOwner

The GarageOwner entity stores accounts for garage business owners who register and manage garages on the GarageGo platform.

Table 5b: GarageOwner Entity Attributes

Attribute	Data Type	Constraint	Description
ownerId (PK)	UUID	NOT NULL, UNIQUE	System-generated surrogate primary key
email	VARCHAR(255)	NOT NULL, UNIQUE	Login email for Garage Owner web application
phone	VARCHAR(20)	NOT NULL, UNIQUE	Contact phone; verified via Twilio OTP
passwordHash	VARCHAR(255)	NOT NULL	BCrypt hashed password (cost factor 12)
isVerified	BOOLEAN	DEFAULT FALSE	TRUE after admin approves first garage registration

Booking

The Booking entity records every scheduled service request from creation through payment, managed by the state machine defined in FR-09.

Table 6: Booking Entity Attributes

Attribute	Data Type	Constraint	Description
bookingId (PK)	UUID	NOT NULL, UNIQUE	System-generated surrogate primary key
customerId (FK)	UUID	NOT NULL	References Customer.customerId
vehicleId (FK)	UUID	NOT NULL	References Vehicle.vehicleId
garageId (FK)	UUID	NOT NULL	References Garage.garageId
serviceType	VARCHAR(100)	NOT NULL	Service requested (e.g., Oil Change, Tyre Replacement)
scheduledAt	TIMESTAMP	NOT NULL	Customer-selected appointment date/time (ISO 8601 UTC)
status	ENUM	NOT NULL	PENDING ACCEPTED IN_SERVICE COMPLETED PAID CANCELLED
estimatedCost	DECIMAL(10,2)	NULLABLE	Pre-service cost estimate in LKR; NULL until garage confirms
finalCost	DECIMAL(10,2)	NULLABLE	Actual cost in LKR; set on status = COMPLETED
paymentStatus	ENUM	NOT NULL, DEFAULT UNPAID	UNPAID PAID – updated by garage owner within 24h (SRS §2.5)
cancellationFee	DECIMAL(10,2)	DEFAULT 0.00	Penalty applied on late cancellation (FR-05 BR-10)
notes	TEXT	NULLABLE	Optional customer notes about the service request
createdAt	TIMESTAMP	NOT NULL	Booking creation timestamp
updatedAt	TIMESTAMP	NOT NULL	Last status transition timestamp

EmergencyRequest

The EmergencyRequest entity records roadside assistance requests triggered by customers via the SOS button, managed through the emergency state machine.

Table 7: EmergencyRequest Entity Attributes

Attribute	Data Type	Constraint	Description
emergencyId (PK)	UUID	NOT NULL, UNIQUE	System-generated surrogate primary key
customerId (FK)	UUID	NOT NULL	References Customer.customerId
vehicleId (FK)	UUID	NOT NULL	References Vehicle.vehicleId
assignedGarageId (FK)	UUID	NULLABLE	References Garage.garageId; NULL until first garage accepts

Attribute	Data Type	Constraint	Description
assignedMechanicId (FK)	UUID	NULLABLE	References Mechanic.mechanicId; NULL until dispatched
customerLatitude	DECIMAL(9,6)	NOT NULL	WGS84 latitude of breakdown location (FR-05)
customerLongitude	DECIMAL(9,6)	NOT NULL	WGS84 longitude of breakdown location (FR-05)
issueDescription	TEXT	NOT NULL	Customer-entered breakdown description
status	ENUM	NOT NULL	SEARCHING ASSIGNED DISPATCHED IN_PROGRESS COMPLETED CANCELLED
searchRadiusKm	INTEGER	DEFAULT 10	Current broadcast radius; expands if no acceptance in 60s (UC-06)
triggeredAt	TIMESTAMP	NOT NULL	Timestamp when SOS button was pressed
assignedAt	TIMESTAMP	NULLABLE	Timestamp of first garage acceptance (FR-06)
completedAt	TIMESTAMP	NULLABLE	Timestamp of mechanic marking job complete

Mechanic

The Mechanic entity records mechanics employed by garages who receive emergency dispatch jobs and share live GPS location.

Table 8: Mechanic Entity Attributes

Attribute	Data Type	Constraint	Description
mechanicId (PK)	UUID	NOT NULL, UNIQUE	System-generated surrogate primary key
garageId (FK)	UUID	NOT NULL	References Garage.garageId (employing garage)
fullName	VARCHAR(150)	NOT NULL	Mechanic's full legal name
phone	VARCHAR(20)	NOT NULL, UNIQUE	Mobile number for OTP login
passwordHash	VARCHAR(255)	NOT NULL	BCrypt hashed password (cost factor 12)
isAvailable	BOOLEAN	DEFAULT TRUE	FALSE = mechanic is currently on an active job
currentLatitude	DECIMAL(9,6)	NULLABLE	Last known GPS latitude (updated every 5s per NFR-P3)
currentLongitude	DECIMAL(9,6)	NULLABLE	Last known GPS longitude
lastLocationAt	TIMESTAMP	NULLABLE	Timestamp of last GPS update

Review

The Review entity stores customer ratings and written feedback for completed garage services.

Table 9: Review Entity Attributes

Attribute	Data Type	Constraint	Description
reviewId (PK)	UUID	NOT NULL, UNIQUE	System-generated surrogate primary key
bookingId (FK)	UUID	NOT NULL, UNIQUE	References Booking.bookingId; one review per completed booking
customerId (FK)	UUID	NOT NULL	References Customer.customerId (must have completed booking)
garageId (FK)	UUID	NOT NULL	References Garage.garageId being reviewed
rating	SMALLINT	NOT NULL, 1–5	Star rating from 1 (poor) to 5 (excellent) per BR-17
reviewText	TEXT	NULLABLE	Optional written review; offensive language auto-rejected (UC-09)
createdAt	TIMESTAMP	NOT NULL	Review submission timestamp

Notification

The Notification entity stores all system-generated push notifications and in-app alerts delivered to Customers, Garage Owners, Mechanics, and Admins via Firebase Cloud Messaging, Web Push API, and Socket.io (FR-14).

Table 9b: Notification Entity Attributes

Attribute	Data Type	Constraint	Description
notificationId (PK)	UUID	NOT NULL, UNIQUE	System-generated primary key
recipientId	UUID	NOT NULL	ID of target user (Customer, GarageOwner, Mechanic, or Admin)
recipientRole	ENUM	NOT NULL	CUSTOMER GARAGE_OWNER MECHANIC ADMIN
type	ENUM	NOT NULL	BOOKING_REQUEST BOOKING_ACCEPTED BOOKING_REJECTED EMERGENCY_BROADCAST EMERGENCY_ASSIGNED MECHANIC_ARRIVED STATUS_UPDATE
payload	JSONB	NOT NULL	JSON body with actionable data (bookingId, emergencyId, message, etc.)
isDelivered	BOOLEAN	DEFAULT FALSE	TRUE once FCM / Web Push / Socket.io confirms delivery
sentAt	TIMESTAMP	NOT NULL	Timestamp when notification was dispatched from the server

Admin

The Admin entity stores administrator accounts with platform-wide oversight capabilities including garage approval, dispute resolution, and user management.

Table 9c: Admin Entity Attributes

Attribute	Data Type	Constraint	Description
adminId (PK)	UUID	NOT NULL, UNIQUE	System-generated surrogate primary key
email	VARCHAR(255)	NOT NULL, UNIQUE	Administrator login email for Admin Dashboard
role	ENUM	NOT NULL	ADMIN (single role for all administrators)

2.3 Key Relationships

- A Customer places zero or more Bookings (1:N); each Booking references exactly one Vehicle and one Garage.
 - A Customer triggers zero or more EmergencyRequests (1:N); each EmergencyRequest references exactly one Vehicle.
 - A Garage is assigned zero or more Bookings (1:N) and zero or more EmergencyRequests (1:N).
 - A Garage employs zero or more Mechanics (1:N); each Mechanic belongs to exactly one Garage.
 - An EmergencyRequest is assigned to at most one Mechanic (1:0..1) after dispatch.
 - A Customer submits at most one Review per completed Booking (1:0..1); each Review updates the Garage's averageRating in real time (FR-13).
-
- A Customer owns zero or more Vehicles (1:N); each Vehicle belongs to exactly one Customer.
 - A GarageOwner manages one or more Garages (1:N); each Garage belongs to exactly one GarageOwner via ownerAccountId (FR-10).
 - Any user role (Customer, GarageOwner, Mechanic, Admin) receives zero or more Notifications (1:N); each Notification targets exactly one recipient identified by recipientId and recipientRole (FR-14).

2.4 Normalization

Table 10: Normalization Analysis (1NF – BCNF)

Entity	1NF	2NF	3NF	BCNF	Notes
Customer	Yes	Yes	Yes	Yes	All attributes depend solely on customerId.
Vehicle	Yes	Yes	Yes	Yes	registrationNo is unique; no partial dependencies.

Entity	1NF	2NF	3NF	BCNF	Notes
Garage	Yes	Yes	Yes	Yes	workingHours stored as JSONB avoids repeating groups.
Booking	Yes	Yes	Yes	Yes	No transitive dependencies; estimatedCost and finalCost are functionally distinct.
EmergencyRequest	Yes	Yes	Yes	Yes	Nullable FKs for assignment enforce sparse population rather than separate tables.
Mechanic	Yes	Yes	Yes	Yes	GPS fields updated in-place; no repeating groups.
Review	Yes	Yes	Yes	Yes	Unique constraint on bookingId prevents multiple reviews per booking.
GarageOwner	Yes	Yes	Yes	Yes	All attributes depend solely on ownerId; no transitive dependencies.
Notification	Yes	Yes	Yes	Yes	payload stored as JSONB avoids repeating groups; recipientRole + recipientId uniquely identify the target.
Admin	Yes	Yes	Yes	Yes	Minimal entity; all attributes depend solely on adminId.

Chapter 3: Class Diagram

This chapter describes the GarageGo domain class model, enumerations, and all UML 2.x relationships with multiplicities. The class diagram is derived directly from the functional requirements (FR-01 to FR-16) and use case descriptions (UC-01 to UC-10) in the GarageGo SRS v0.1.

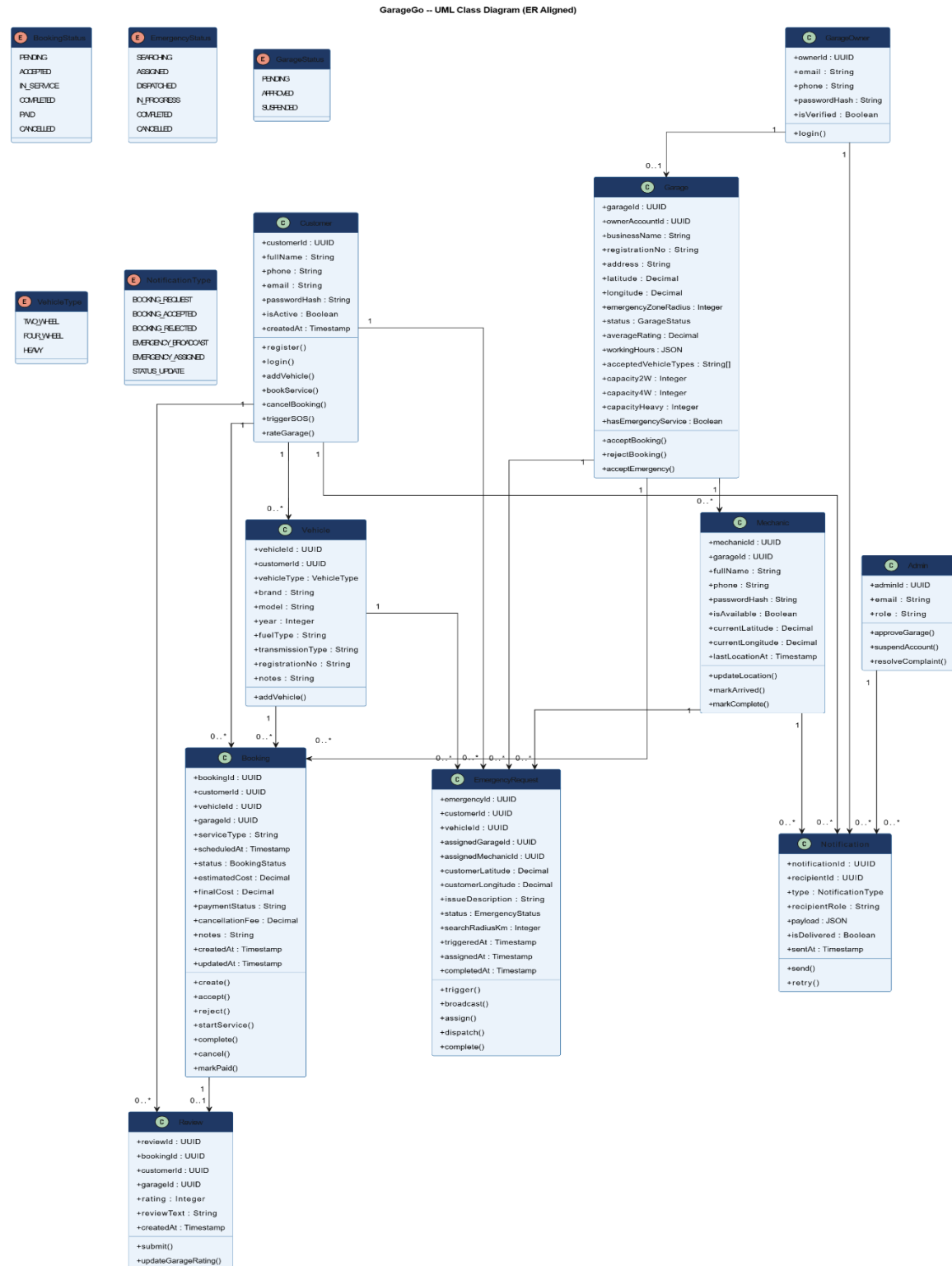


Figure 3: GarageGo – UML Class Diagram [see documents/diagrams/class-diagram/ClassDiagram.png]

3.1 Domain Class Descriptions

Table 11: Domain Class Descriptions

Class	Key Attributes	Key Methods	Source FR / UC
Customer	customerId, fullName, phone, email, passwordHash, nicNumber, isActive	register(), login(), addVehicle(), triggerSOS(), bookService(), cancelBooking(), rateGarage()	FR-01, UC-01, UC-06
Vehicle	vehicleId, customerId, vehicleType, brand, model, registrationNo	addVehicle(), getServiceHistory()	FR-02, UC-02
Garage	garageId, businessName, status, averageRating, capacity2W, capacity4W, hasEmergencyService	register(), updateCapacity(), acceptBooking(), rejectBooking(), acceptEmergency()	FR-08, FR-10, UC-10
GarageOwner	ownerId, businessEmail, phone, isVerified	login(), assignMechanic(), generateInvoice(), viewAnalytics()	FR-10, UC-10
Mechanic	mechanicId, garageId, fullName, isAvailable, currentLatitude, currentLongitude	login(), updateLocation(), markArrived(), markComplete()	FR-07, FR-15, UC-07
Booking	bookingId, customerId, vehicleId, garageId, status, scheduledAt, estimatedCost, finalCost	create(), accept(), reject(), startService(), complete(), cancel(), markPaid()	FR-04, FR-08, FR-09
EmergencyRequest	emergencyId, customerId, vehicleId, status, customerLatitude, customerLongitude, searchRadiusKm	trigger(), broadcastToGarages(), assignFirstAccept(), expandRadius(), dispatch(), complete()	FR-05, FR-06, FR-07
Review	reviewId, bookingId, customerId, garageId, rating, reviewText	submit(), validateContent(), updateGarageRating()	FR-13, UC-09
Admin	adminId, email, role	approveGarage(), suspendAccount(), resolveComplaint(), viewAnalytics()	FR-11, UC-21
Notification	notificationId, recipientId, type, payload, sentAt	send(), retry()	FR-14

3.2 Enumerations

Table 12: Enumerations

Enumeration	Values	Used In
VehicleType	TWO_WHEEL, FOUR_WHEEL, HEAVY	Vehicle, Garage (capacity fields), EmergencyRequest
BookingStatus	PENDING, ACCEPTED, IN_SERVICE, COMPLETED, PAID, CANCELLED	Booking (state machine FR-09)
EmergencyStatus	SEARCHING, ASSIGNED, DISPATCHED, IN_PROGRESS, COMPLETED, CANCELLED	EmergencyRequest (state machine FR-05, FR-06)
GarageStatus	PENDING, APPROVED, SUSPENDED	Garage (approval lifecycle FR-10, FR-11)
PaymentStatus	UNPAID, PAID	Booking (manual confirmation per SRS §2.5)
NotificationType	BOOKING_REQUEST, BOOKING_ACCEPTED, BOOKING_REJECTED, EMERGENCY_BROADCAST, EMERGENCY_ASSIGNED, MECHANIC_ARRIVED, STATUS_UPDATE	Notification (FR-14)
UserRole	CUSTOMER, GARAGE_OWNER, MECHANIC, ADMIN	All user entities (RBAC enforcement NFR-S1)
FuelType	PETROL, DIESEL, ELECTRIC, HYBRID	Vehicle
TransmissionType	MANUAL, AUTOMATIC	Vehicle

3.3 Relationships and Multiplicity

Table 13: UML Relationships and Multiplicity

Relationship	Type	Multiplicity	Description
Customer → Vehicle	Composition	1 : 0..*	A Customer owns zero or more Vehicles; Vehicles are deleted when the Customer is deleted.
Customer → Booking	Association	1 : 0..*	A Customer places zero or more Bookings.
Customer → EmergencyRequest	Association	1 : 0..*	A Customer triggers zero or more EmergencyRequests.
Vehicle → Booking	Association	1 : 0..*	A Vehicle appears in zero or more Bookings.
Garage → GarageOwner	Association	1 : 1	A Garage is managed by exactly one GarageOwner.

Relationship	Type	Multiplicity	Description
Garage → Mechanic	Aggregation	1 : 1..*	A Garage employs one or more Mechanics.
Garage → Booking	Association	1 : 0..*	A Garage fulfils zero or more Bookings.
EmergencyRequest → Garage	Association	0..* : 0..1	An EmergencyRequest is assigned to at most one Garage after first-accept-wins.
EmergencyRequest → Mechanic	Association	1 : 0..1	An assigned EmergencyRequest is dispatched to at most one Mechanic.
Booking → Review	Association	1 : 0..1	A completed Booking may have at most one Review submitted.
Review → Garage	Association	0..* : 1	Multiple Reviews aggregate to update a Garage's averageRating.
Notification → Customer / GarageOwner / Mechanic	Dependency	1 : 1	Each Notification targets exactly one recipient across user roles.

Chapter 4: Sequence Diagrams

This chapter presents sequence diagrams for the use cases defined in the GarageGo SRS v0.1 (UC-01 to UC-10). Each diagram shows internal object interactions, method call labels, return values, loop frames, and alternative frames in UML 2.x notation. Diagrams are stored in documents/diagrams/sequence-diagram/.

4.1 UC-01 – Register Account

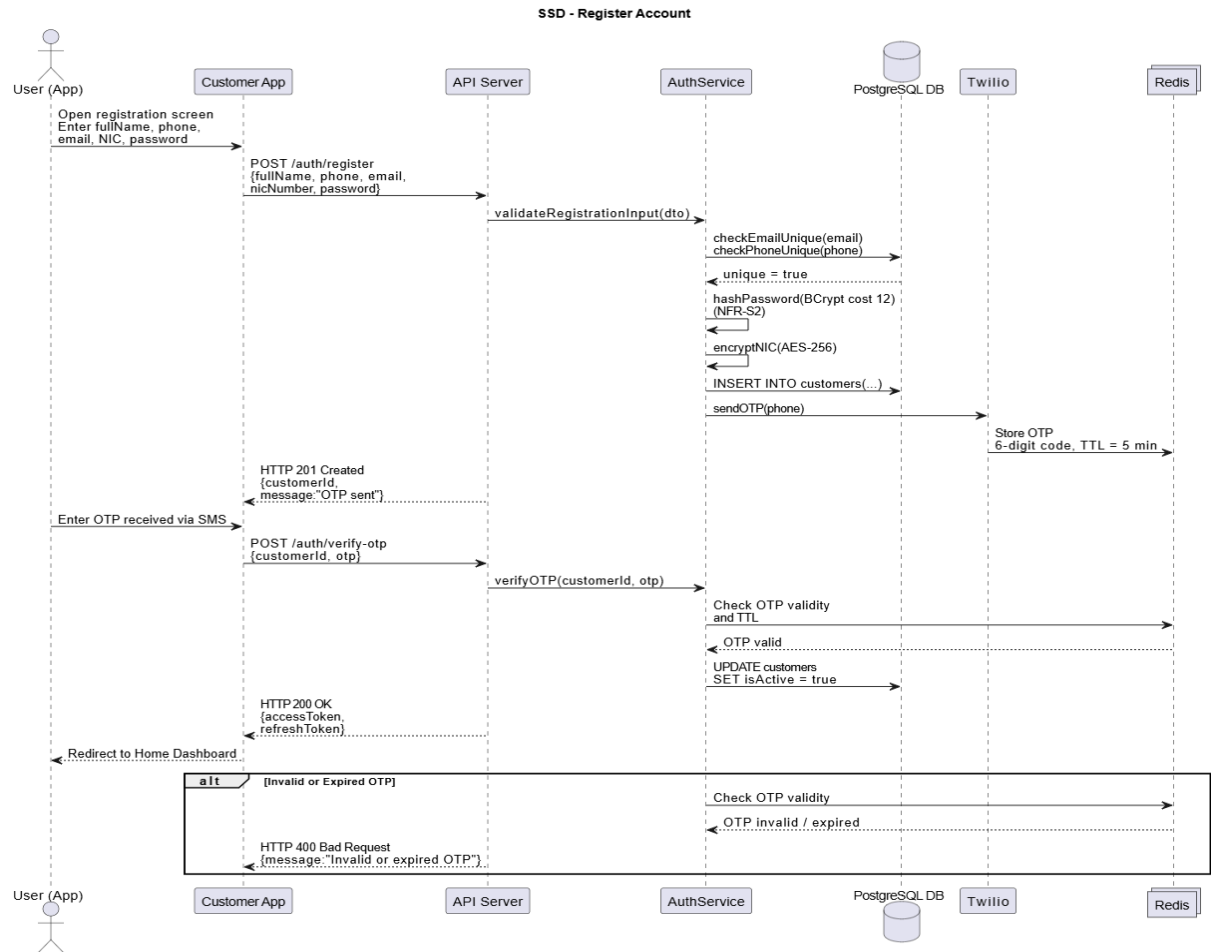


Figure 4: Sequence Diagram – Register Account [see documents/diagrams/sequence-diagram/RegisterAccount.png]

Table 14: Sequence Steps – UC-01 Register Account

#	From	To	Message / Action
1	User (App)	Customer App	Open registration screen; enter fullName, phone, email, NIC, password
2	Customer App	POST /auth/register	{ fullName, phone, email, nicNumber, password }
3	API Server	AuthService	validateRegistrationInput(dto)

#	From	To	Message / Action
4	AuthService	PostgreSQL DB	SELECT checkEmailUnique(email), checkPhoneUnique(phone)
5	PostgreSQL DB	AuthService	return: unique = true
6	AuthService	AuthService	hashPassword(BCrypt cost 12) [self-call] (NFR-S2)
7	AuthService	AuthService	encryptNIC(AES-256) [self-call]
8	AuthService	PostgreSQL DB	INSERT INTO customers (...)
9	AuthService	Twilio	sendOTP(phone) – 6-digit OTP with 5-min TTL stored in Redis
10	API Server	Customer App	HTTP 201 Created { customerId, message: 'OTP sent' }
11	User (App)	Customer App	Enter OTP received via SMS
12	Customer App	POST /auth/verify-otp	{ customerId, otp }
13	API Server	AuthService	verifyOTP(customerId, otp) — checks Redis TTL
14	AuthService	PostgreSQL DB	UPDATE customers SET isActive = true
15	API Server	Customer App	HTTP 200 OK { accessToken, refreshToken }
16	Customer App	User (App)	Redirect to Home Dashboard

4.2 UC-04 – Book Service

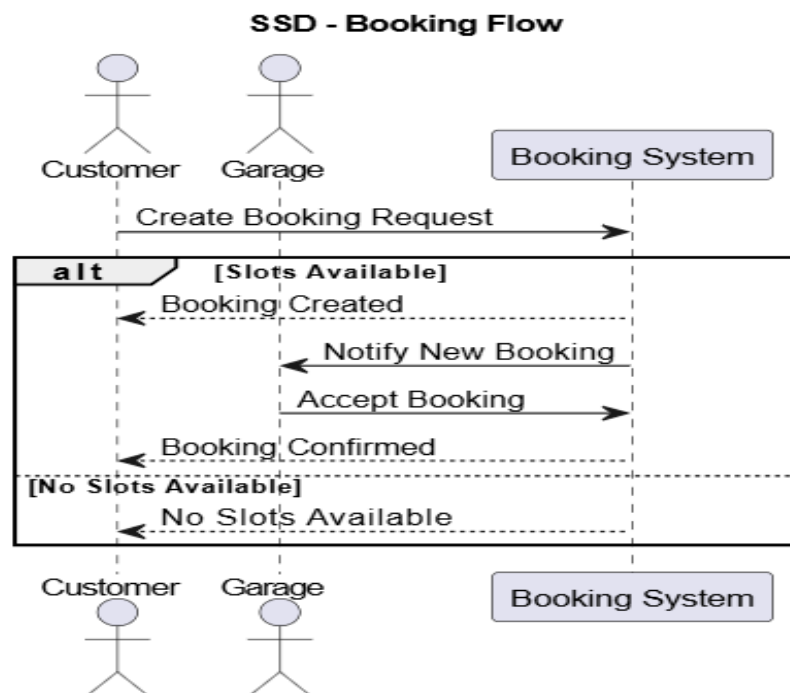


Figure 5: Sequence Diagram – Book Service [see documents/diagrams/sequence-diagram/BookingFlow_Final.png]

Table 15: Sequence Steps – UC-04 Book Service

#	From	To	Message / Action
1	Customer	Customer App	Select garage, vehicle, service type, date/time; add optional notes
2	Customer App	POST /bookings	{ garageId, vehicleId, serviceType, scheduledAt, notes }
3	API Server	BookingService	validateBookingRequest(dto)
4	BookingService	PostgreSQL DB	SELECT checkCapacity(garageId, vehicleType, scheduledAt)
5	PostgreSQL DB	BookingService	return: available = true (slot available)
ALT	Slot unavailable		PostgreSQL returns available = false → API returns HTTP 409 Conflict
6	BookingService	PostgreSQL DB	INSERT INTO bookings (...) status = PENDING
7	BookingService	NotificationService	sendPushNotification(garageOwnerId, 'NEW_BOOKING_REQUEST')
8	NotificationService	FCM	POST /fcm/send { recipientToken, payload }
9	API Server	Customer App	HTTP 201 Created { bookingId, status: PENDING }
10	Customer App	Customer	Display 'Booking submitted – awaiting garage confirmation'
11	GarageOwner App	POST /bookings/{id}/accept	Garage owner taps Accept within 2-hour window (BR-08)
12	API Server	BookingService	transitionStatus(bookingId, PENDING → ACCEPTED)
13	BookingService	PostgreSQL DB	UPDATE bookings SET status = ACCEPTED
14	BookingService	NotificationService	sendPushNotification(customerId, 'BOOKING_ACCEPTED')
15	API Server	Customer App	Socket.io event: booking_accepted { bookingId }

4.3 UC-06 & UC-07 – Emergency SOS Dispatch and Mechanic GPS Tracking

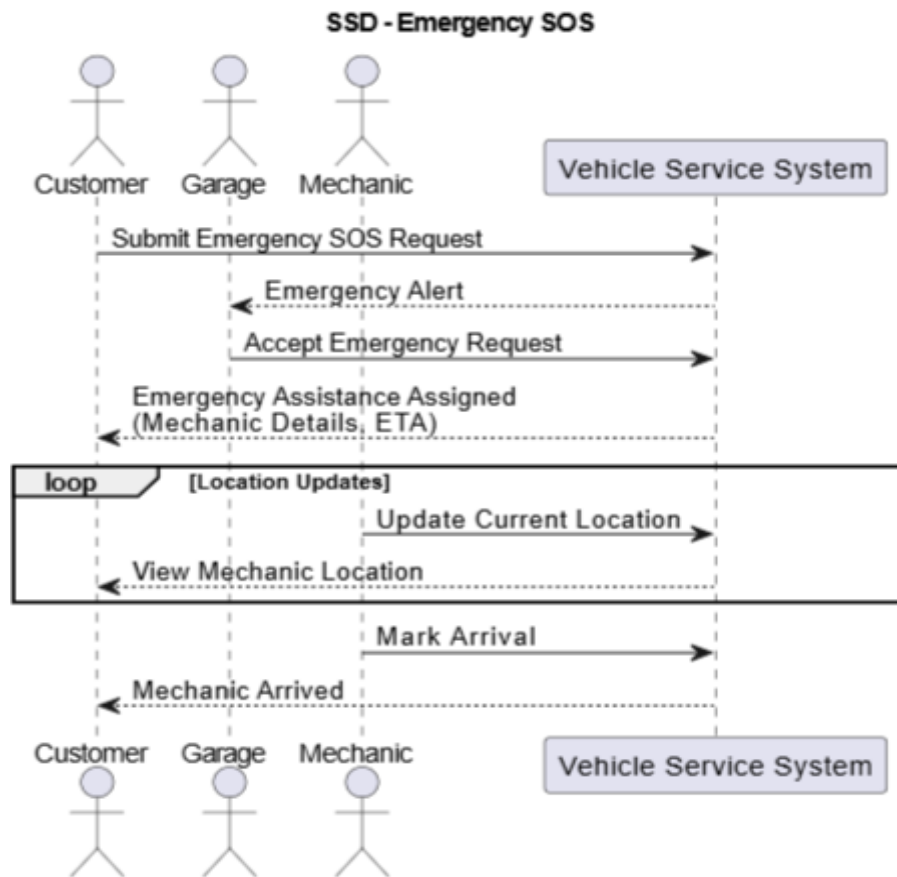


Figure 6: Sequence Diagram – Emergency SOS Full Flow [see documents/diagrams/sequence-diagram/EmergencySOS_Final.png]

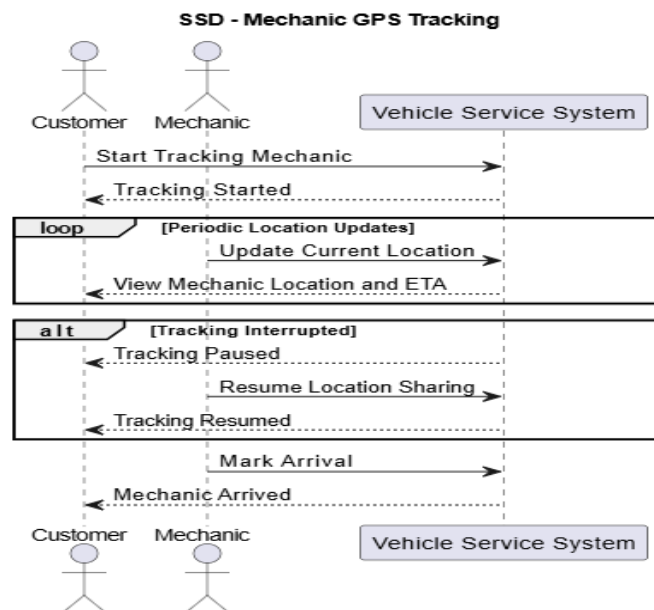


Figure 7: Sequence Diagram – Mechanic Tracking [see documents/diagrams/sequence-diagram/MechanicTracking_Final.png]

Table 16: Sequence Steps – UC-06 Emergency SOS Dispatch

#	From	To	Message / Action
1	Customer	Customer App	Press SOS button; select vehicle; enter issue description
2	Customer App	POST /emergency	{ vehicleId, issueDescription, latitude, longitude }
3	API Server	EmergencyService	createEmergencyRequest(dto)
4	EmergencyService	PostgreSQL DB	INSERT INTO emergency_requests (...) status = SEARCHING
5	EmergencyService	Google Maps API	findNearbyGarages(latitude, longitude, radius=10km)
6	Google Maps API	EmergencyService	return: [garageId1, garageId2, garageId3]
7	EmergencyService	Socket.io Server	broadcast('emergency_alert', payload) to all eligible garage rooms
8	API Server	Customer App	HTTP 201 { emergencyId, status: SEARCHING }
LOOP	60-second acceptance window		
9	GarageOwner App	POST /emergency/{id}/accept	First garage owner taps Accept
10	API Server	EmergencyService	assignFirstAccept(emergencyId, garageId) — SELECT FOR UPDATE lock (FR-06)
11	EmergencyService	PostgreSQL DB	UPDATE emergency_requests SET status=ASSIGNED, assignedGarageId=garageId
12	EmergencyService	Socket.io Server	emit('emergency_taken') to all other eligible garages
13	EmergencyService	NotificationService	sendPushNotification(customerId, 'EMERGENCY_ASSIGNED')
ALT	No acceptance in 60s		EmergencyService.expandRadius(emergencyId, +10km); repeat broadcast
14	GarageOwner App	POST /emergency/{id}/dispatch	{ mechanicId } – garage assigns mechanic
15	API Server	EmergencyService	dispatchMechanic(emergencyId, mechanicId)
16	EmergencyService	PostgreSQL DB	UPDATE emergency_requests SET status=DISPATCHED, assignedMechanicId
17	EmergencyService	Mechanic App (Socket.io)	emit('job_assigned', { customerLocation, vehicleInfo })
LOOP	GPS tracking loop — every 5 seconds (NFR-P3)		
18	Mechanic App	Socket.io Server	emit('location_update', { lat, lng, emergencyId })
19	Socket.io Server	EmergencyService	updateMechanicLocation(mechanicId, lat, lng)
20	EmergencyService	PostgreSQL DB	UPDATE mechanics SET currentLatitude, currentLongitude, lastLocationAt

#	From	To	Message / Action
21	Socket.io Server	Customer App	emit('mechanic_location', { lat, lng, eta }) within 1s (NFR-P3)
22	Customer App	Customer	Display mechanic pin moving on Google Maps with live ETA
ALT	Signal loss > 30s		Customer App displays 'Tracking paused – reconnecting...' (FR-07)

4.4 UC-10 – Register Garage and UC-21 – Admin Garage Approval

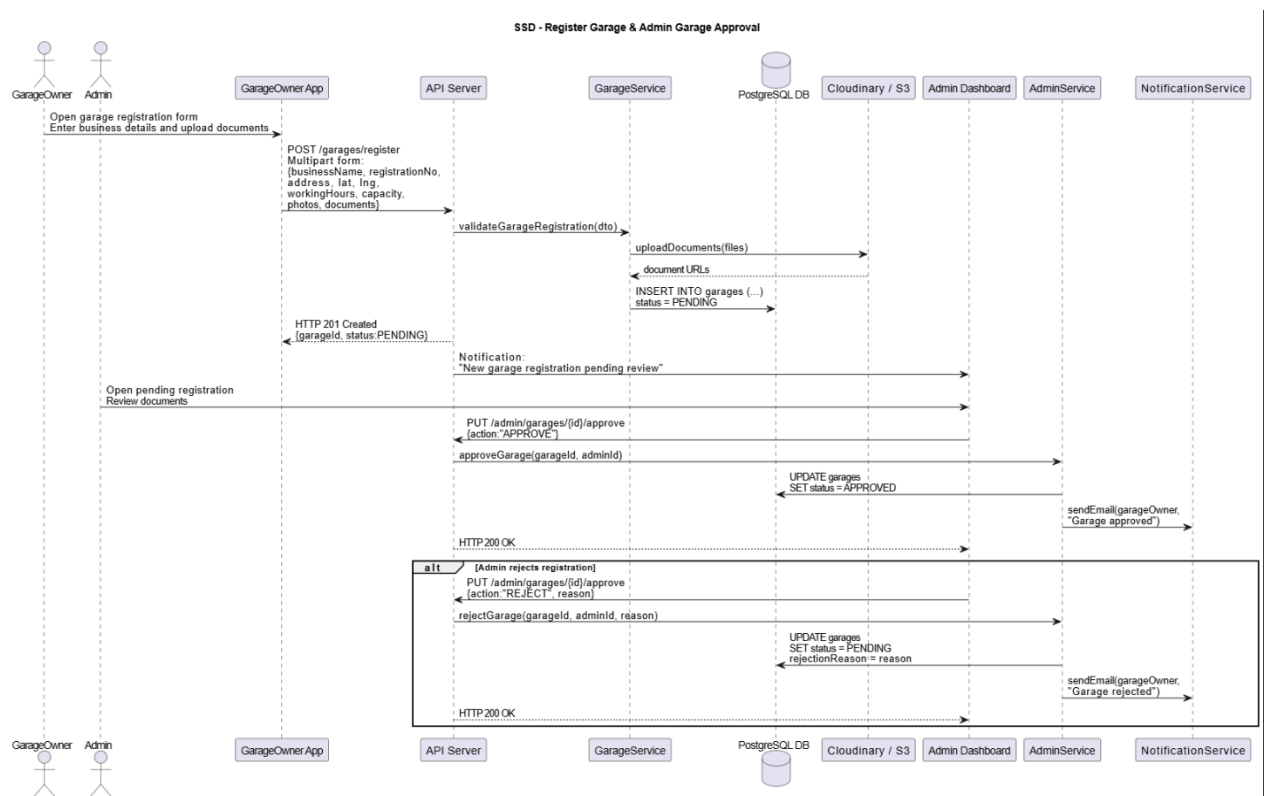


Figure 8: Sequence Diagram – Register Garage & Admin Approval [see documents/diagrams/sequence-diagram/GarageRegistration&Approval.png]

Table 17: Sequence Steps – UC-10 Register Garage and UC-21 Admin Approval

#	From	To	Message / Action
1	GarageOwner	GarageOwner App	Open garage registration form; enter all business details and upload documents
2	GarageOwner App	POST /garages/register	Multipart form: { businessName, registrationNo, address, lat, lng, workingHours, capacity, photos, documents }
3	API Server	GarageService	validateGarageRegistration(dto)

#	From	To	Message / Action
4	GarageService	Cloudinary / S3	uploadDocuments(files) – returns document URLs
5	GarageService	PostgreSQL DB	INSERT INTO garages (...) status = PENDING
6	API Server	GarageOwner App	HTTP 201 Created { garageId, status: PENDING }
7	API Server	Admin Dashboard	Notification: 'New garage registration pending review'
8	Admin	Admin Dashboard	Open pending garage registration; review documents
9	Admin Dashboard	PUT /admin/garages/{id}/approve	{ action: 'APPROVE' }
10	API Server	AdminService	approveGarage(garageId, adminId)
11	AdminService	PostgreSQL DB	UPDATE garages SET status = APPROVED
12	AdminService	NotificationService	sendEmail(garageOwner, 'Garage approved')
13	API Server	Admin Dashboard	HTTP 200 OK
ALT	Admin rejects registration		AdminService updates status = PENDING with reason; GarageOwner receives rejection email

Chapter 5: State Machine Diagrams

This chapter presents state machine diagrams for the two primary stateful entities in the GarageGo system: Booking and EmergencyRequest. Both entities have well-defined lifecycle states whose transitions are governed by business rules defined in the SRS. These diagrams directly realize FR-09 (Booking State Machine) and FR-05/FR-06 (Emergency Request lifecycle).

5.1 Booking State Machine

The Booking entity progresses through six states as defined in FR-09. The cancelled state is reachable from both PENDING and ACCEPTED, subject to the cancellation policy defined in UC-05.

Figure 9: GarageGo – Booking State Machine Diagram [see documents/diagrams/SM-Booking.png]

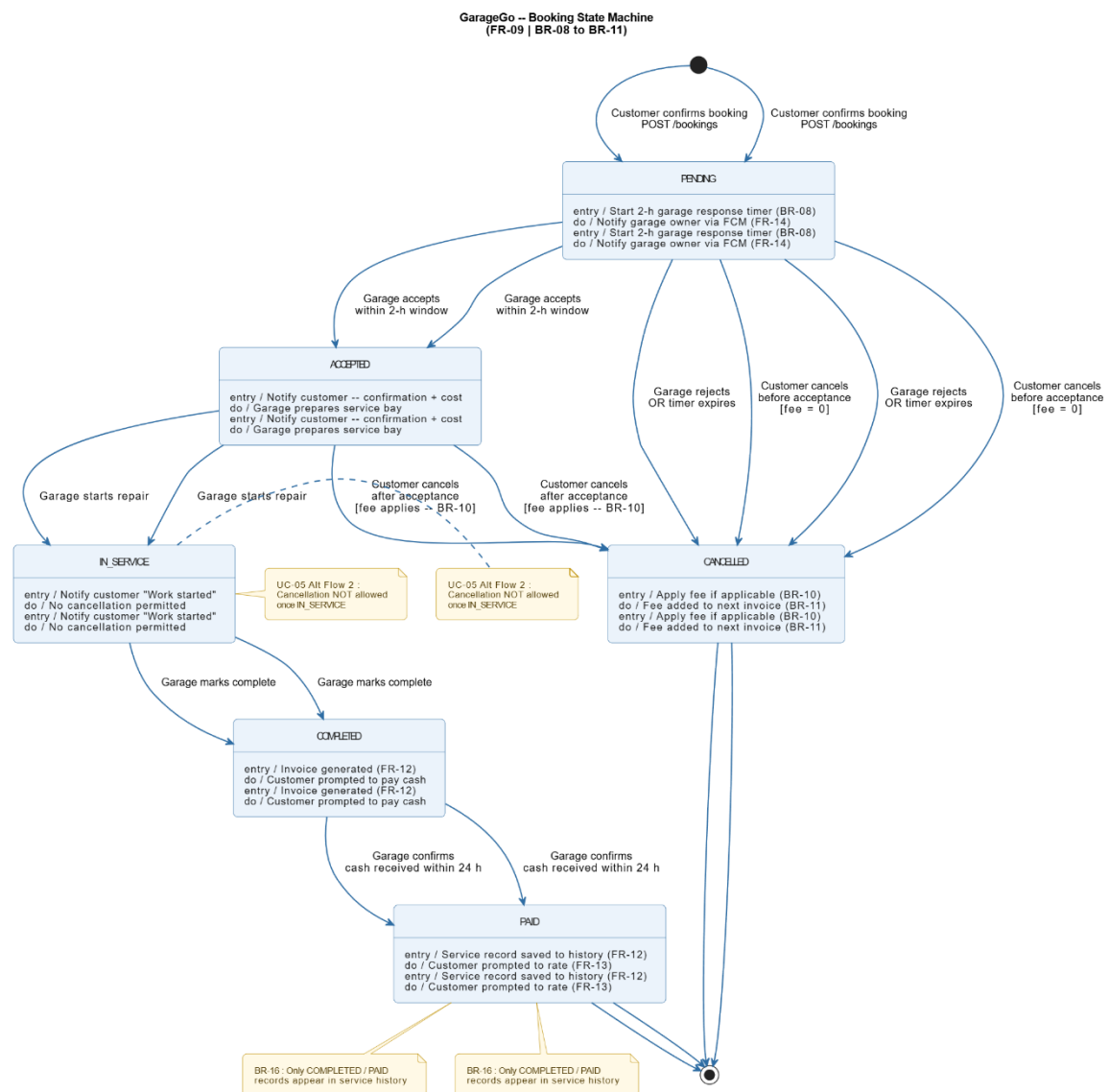


Table 18: Booking State Transitions

From State	Trigger / Event	To State	Business Rule / FR Reference
[Initial]	Customer confirms booking request	PENDING	FR-04; booking stored with status PENDING; 2-hour garage response timer starts (BR-08)
PENDING	Garage owner accepts booking	ACCEPTED	FR-09; push notification sent to customer (FR-14)
PENDING	Garage owner rejects booking / timer expires	CANCELLED	BR-08; customer notified of rejection
PENDING	Customer cancels before acceptance	CANCELLED	UC-05; no cancellation fee if not yet accepted (BR-10)
ACCEPTED	Customer cancels after acceptance	CANCELLED	UC-05 Alt Flow 1; cancellation fee applied (BR-10)
ACCEPTED	Garage starts the repair work	IN_SERVICE	FR-09; cannot cancel once in service
IN_SERVICE	Garage marks repair complete	COMPLETED	FR-09; final cost recorded; customer prompted to pay
COMPLETED	Garage owner confirms cash receipt	PAID	SRS §2.5; payment confirmed within 24 hours
PAID	[Terminal]	—	Service history updated; customer prompted for review (FR-13)
CANCELLED	[Terminal]	—	Cancellation fee (if any) added to next invoice (BR-11)

5.2 EmergencyRequest State Machine

The EmergencyRequest entity progresses through six states from the moment a customer presses the SOS button to the completion of roadside assistance. The search radius expansion mechanism is a guard-triggered automatic transition if no garage accepts within 60 seconds.

Figure 10: GarageGo – EmergencyRequest State Machine Diagram [see documents/diagrams//SM-EmergencyRequest.png]

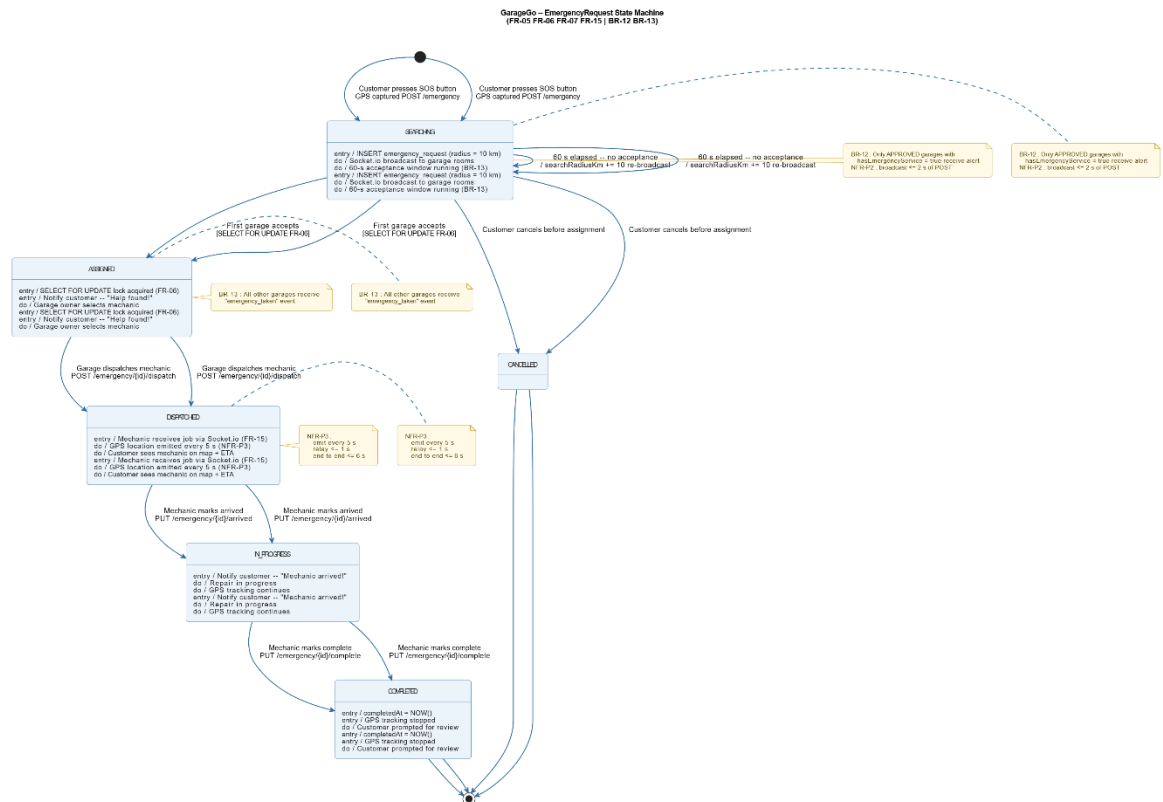


Table 20: EmergencyRequest State Transitions

From State	Trigger / Event	To State	Business Rule / FR Reference
[Initial]	Customer presses SOS button; GPS captured	SEARCHING	FR-05; broadcast sent to all eligible garages within radius (BR-12)
SEARCHING	First garage accepts within 60 seconds	ASSIGNED	FR-06; SELECT FOR UPDATE lock prevents race condition; other garages notified (BR-13)
SEARCHING	60 seconds elapsed with no acceptance	SEARCHING	UC-06 Alt Flow 2; searchRadiusKm += 10; new broadcast sent
SEARCHING	Customer cancels before assignment	CANCELLED	UC-06 cancellation path
ASSIGNED	Garage owner assigns and dispatches mechanic	DISPATCHED	FR-15; mechanic app receives job details
DISPATCHED	Mechanic marks arrived at customer location	IN_PROGRESS	Mechanic app 'Mark Arrived' action; customer notified (FR-14)

From State	Trigger / Event	To State	Business Rule / FR Reference
IN_PROGRESS	Mechanic marks job complete	COMPLETED	Mechanic app 'Mark Complete' action; service record created
COMPLETED	[Terminal]	—	Customer prompted to review garage; emergency closes
CANCELLED	[Terminal]	—	No charge applied; customer may trigger new SOS

Chapter 6: UI/UX Design

This chapter describes the user interface design principles, key screen descriptions with accessibility considerations, and the navigation flow for the GarageGo Customer App and Garage Owner App. All screens are implemented using React Native (Expo) with responsive layouts targeting iOS 15+ and Android 10+.

6.1 Design Principles

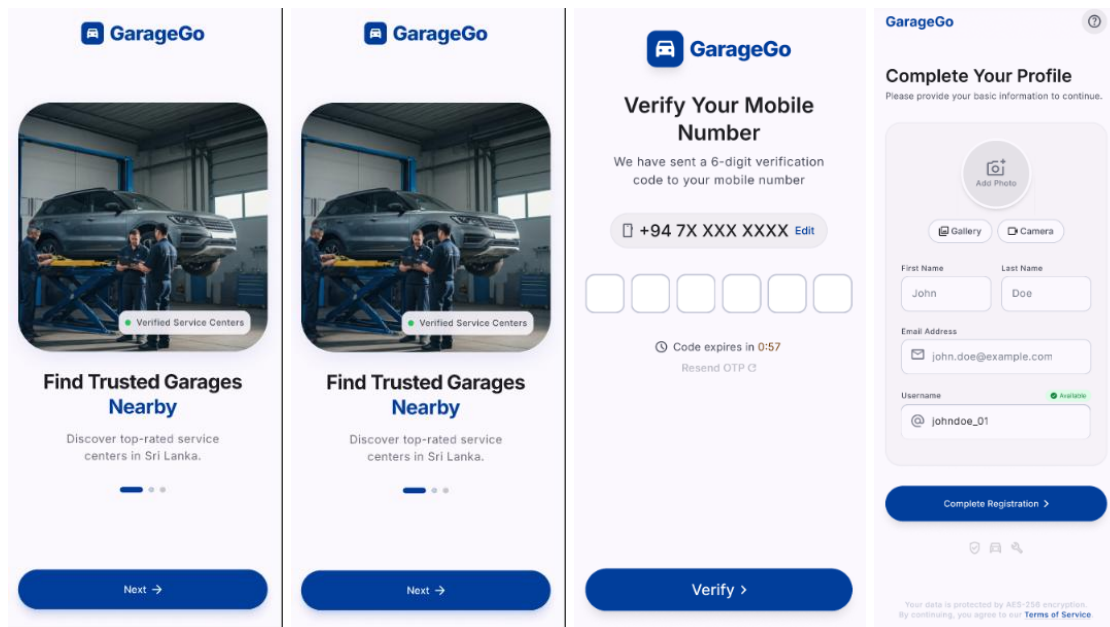
- **Emergency First:** The SOS button is the most prominent element on the Customer App home screen, visible without scrolling at a minimum tap target of 48×48 dp (NFR-U1), coloured red to signal urgency.
- **Transparency:** All garage listings display pricing, availability, and ratings upfront, directly addressing the price opacity pain point identified in Interview #9 (Persona Kasun: 'I want to see the price before they start working').
- **Feedback:** Every user action receives immediate visual feedback including loading indicators, success toasts, and error banners. Offline state is communicated within 3 seconds (NFR-U3).
- **Simplicity:** Each screen focuses on a single primary task; secondary actions are placed in bottom sheets or secondary panels to reduce cognitive load for users in stressful breakdown situations.
- **Accessibility:** Minimum tap target of 48×48 dp throughout (Material Design guidelines); ARIA-equivalent React Native accessibility labels on all interactive components; colour contrast ratio meeting WCAG 2.1 AA.
- **Registration Speed:** The full registration flow is achievable within 3 minutes on a standard 4G connection (NFR-U2), with OTP auto-detection where supported by the OS.

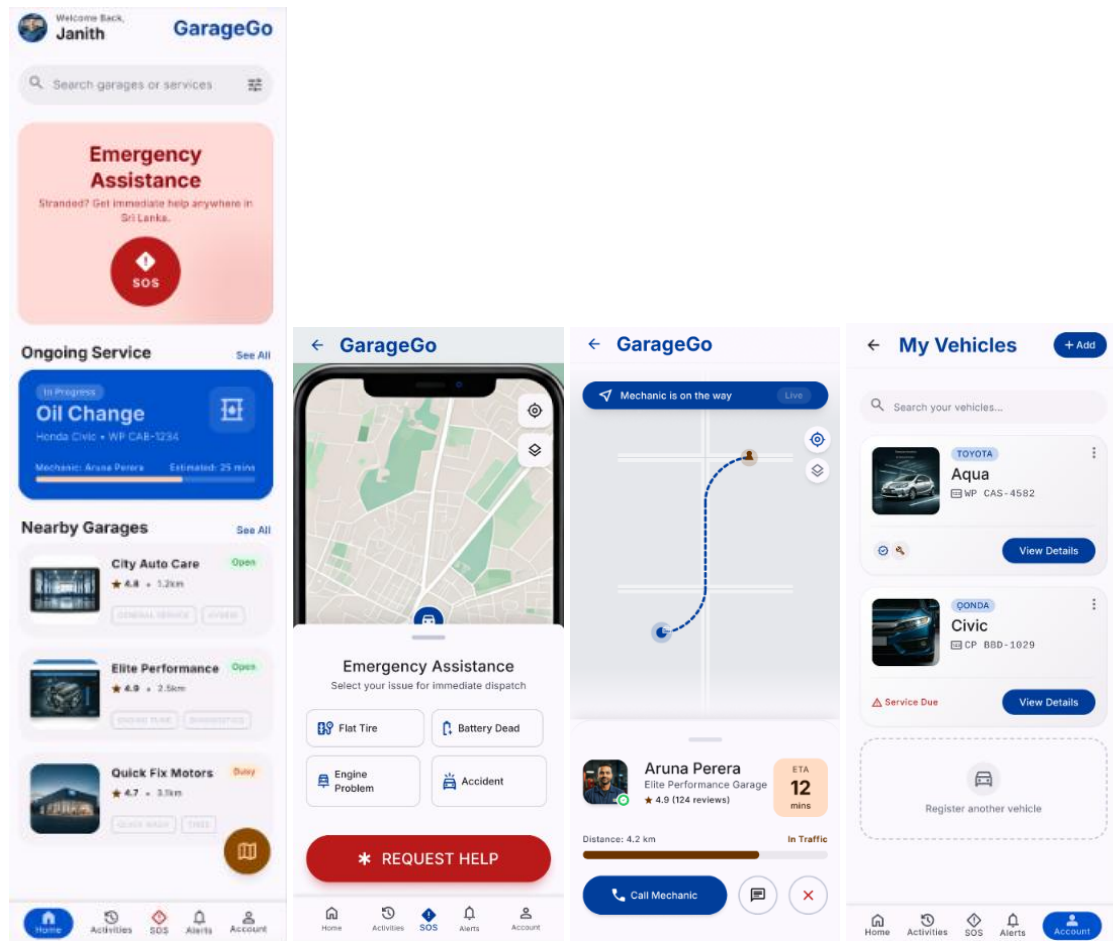
6.2 Screen Descriptions and Accessibility Considerations

Table 21: Customer App – Screen Descriptions and Accessibility

Screen	Key UI Elements	Accessibility Considerations
Registration / OTP	Phone field, email field, NIC field, password field, OTP input (6-digit)	All inputs have accessibilityLabel; OTP field auto-focuses on SMS delivery; password field has show/hide toggle; error states described by accessibilityHint
Home Dashboard	Red SOS button (48×48 dp minimum, full-width on mobile), quick-action tiles (Book Service, My Vehicles, History), notification bell	SOS button has accessibilityRole='button' and accessibilityLabel='Emergency SOS'; notification badge count exposed via accessibilityLabel; visible focus ring on all tappable elements
Garage Search & Map	Google Maps view with garage pins, list toggle, filter panel (vehicle type, rating, distance, services), availability badge per garage card	Filter panel keyboard-navigable; garage pins have accessibilityLabel with garage name and rating; map has text-based list alternative
Garage Profile	Garage photos, star rating, review list, service menu with prices, working hours, 'Book Service' CTA	Images have descriptive alt text; star rating exposed as numeric value via

Screen	Key UI Elements	Accessibility Considerations
		accessibilityValue; CTA button minimum 48dp height
Booking Creation	Vehicle selector, service type picker, date/time picker, notes field, estimated cost display, 'Confirm Booking' button	Step progress indicator with accessibilityRole='progressbar'; date picker uses native OS picker for accessibility; estimated cost announced on change
Emergency SOS Trigger	Full-screen modal: vehicle selector, text area for issue description, GPS location pin with manual adjust option, 'Send SOS' button	Critical flow: full-screen removes distractions; GPS status announced via accessibilityLiveRegion; manual location entry available if GPS unavailable (UC-06 Alt Flow 1)
Mechanic Live Tracking	Google Maps with animated mechanic marker, ETA banner, status timeline (Dispatched > On the Way > Arrived), 'Call Mechanic' button	Text-based ETA panel for non-visual users; status changes announced via accessibilityLiveRegion='polite'; 'Call Mechanic' uses tel: URI for native phone dialler
Service History	Vehicle-grouped list of past bookings; each card shows garage, date, services, cost, payment status; invoice download button	List uses accessibilityRole='list'; each item announces key details; invoice download button labeled with specific booking date
Rate & Review	1–5 star selector, optional text area, 'Submit' button	Star selector implements accessibilityRole='slider' with accessibilityValue; text area has 300 character counter announced on change

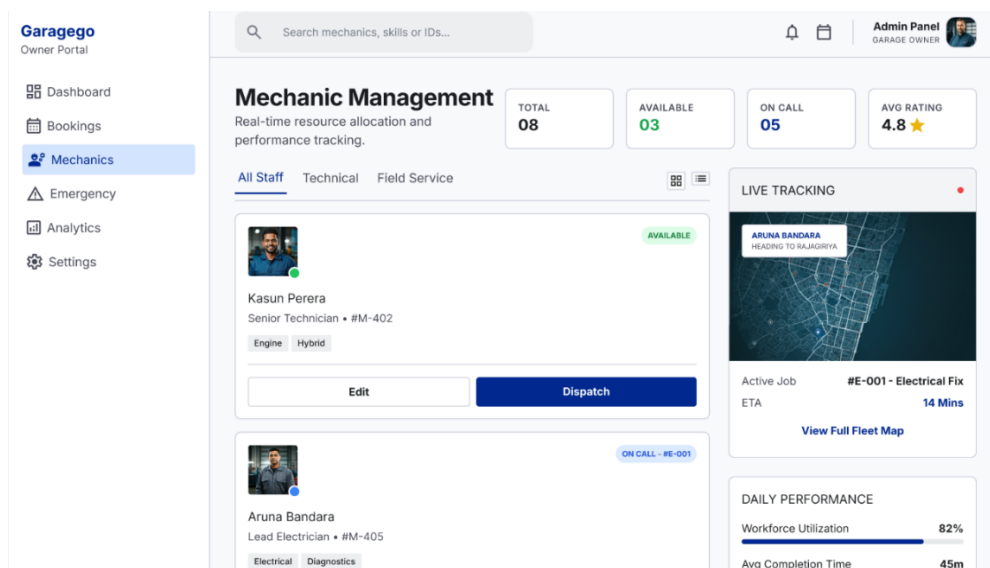
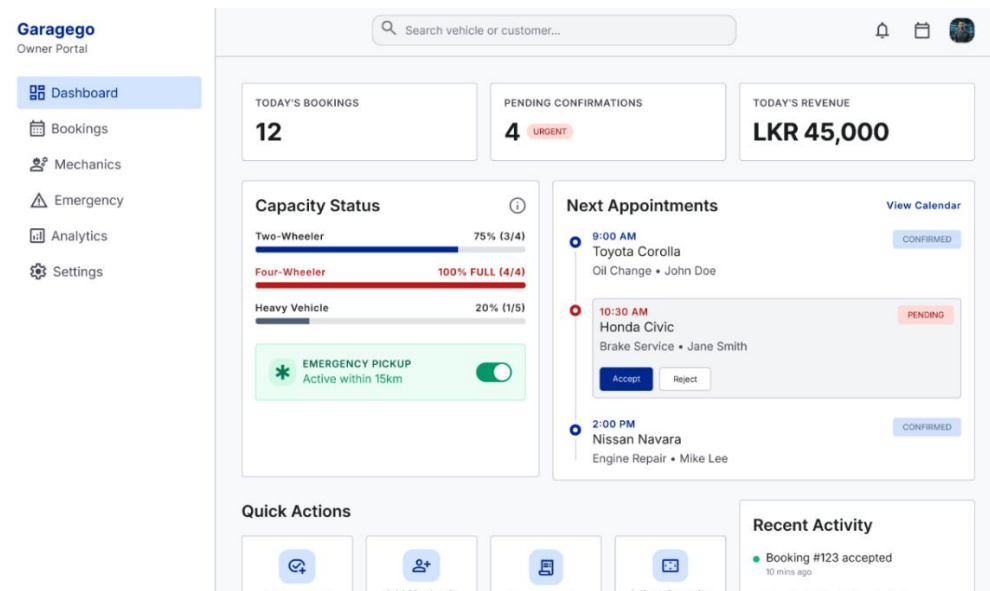
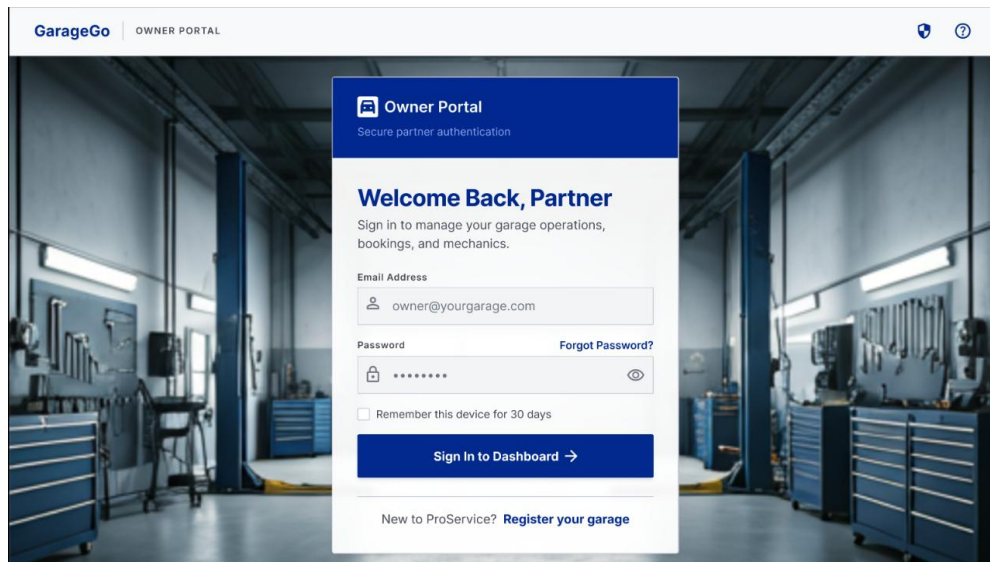




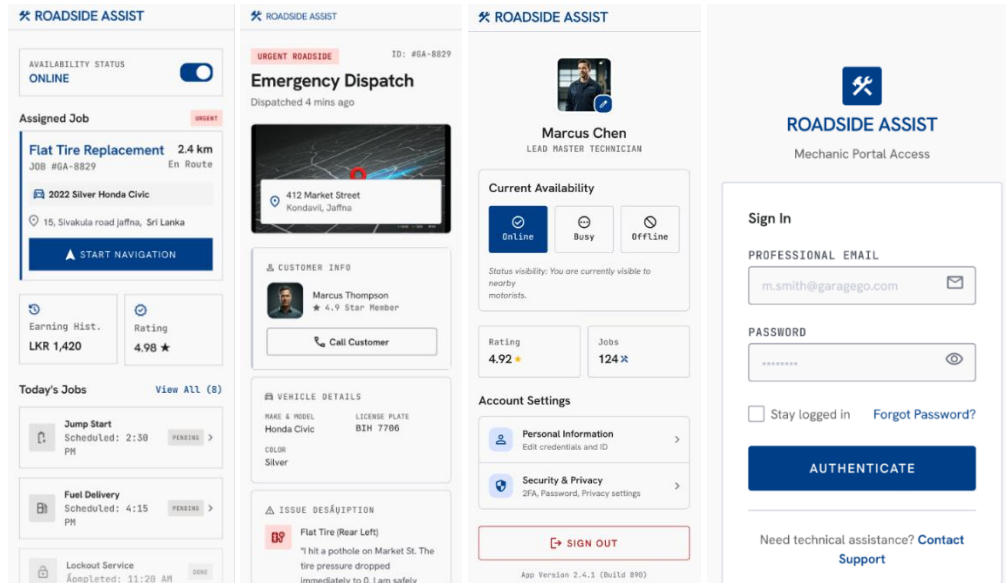
Untitled – Figma

Table 22: Garage Owner App – Screen Descriptions

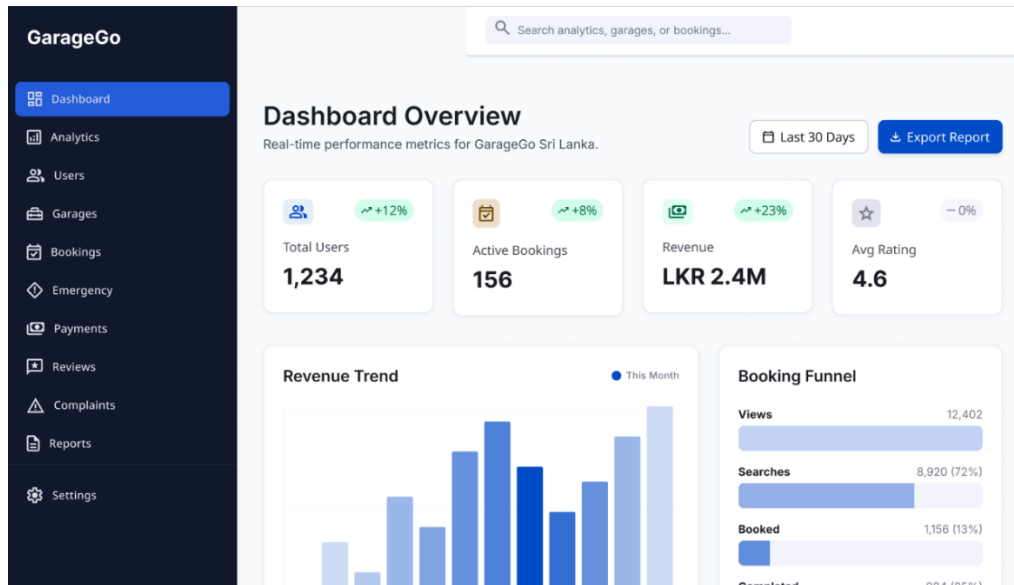
Screen	Key UI Elements	Accessibility Considerations
Dashboard	Active bookings count, capacity gauge per vehicle type, incoming requests badge, revenue summary	Capacity gauges expose numeric value via accessibilityValue; incoming request badge announced via accessibilityLiveRegion
Booking Request	Booking detail card (customer, vehicle, service, time), 'Accept' and 'Reject' buttons, estimated cost input, 2-hour countdown timer	Timer announced at 30-minute intervals via accessibilityLiveRegion; Accept/Reject buttons minimum 48dp; cost input validated with inline error
Emergency Alert	Full-screen modal: customer location pin, vehicle info, issue description, distance from garage, 'Accept Emergency' button	High-contrast alert background; distance announced; 60-second countdown timer with accessibilityLiveRegion
Mechanic Assignment	List of available mechanics, 'Assign' button per mechanic, confirmation dialog	Mechanic list uses accessibilityRole='list'; confirmation dialog uses alertdialog role
Service Management	Active bookings list with status controls: Start Service, Complete Service, Generate Invoice	Status transition buttons labeled with next state; invoice generation confirms action via alert



Garage mechanic App – Screen



Admin App – Screen



GarageGo

Admin Portal

Dashboard

Analytics

Users

Garages

Bookings

Emergency

Payments

Reviews

Complaints

Reports

Settings

Search by name, phone or NIC...

GarageGo Admin

User Management

Manage all participants in the GarageGo ecosystem.

Export

Add User

All Users

Customers1,024

Garage Owners156

Mechanics48

Admins6

Role: All

Status: All

Verification: All

Clear all filters

	NAME	ROLE	CONTACT INFO	NIC STATUS	STATUS	ACTIONS
☐	Kasun Wijesinghe Joined: 12 May 2023	Garage Owner	+94 77 123 4567 kasun.w@gmail.com	Verified	Active	👁️✎️🚫
☐	Aruni Perera Joined: 04 Jan 2024	Customer	+94 71 456 7890 aruni.p@yahoo.com	Verified	Active	👁️✎️🚫
☐	Sunil Mendis Joined: 15 Aug 2023	Mechanic	+94 72 987 6543 sunil.m@garagego.lk	Pending	Pending	👁️✎️🚫

GarageGo

Admin Portal

Dashboard

Analytics

Users

Garages

Bookings

Emergency

Payments

Reviews

Complaints

Reports

Settings

Complaints & Disputes

Mediating and resolving marketplace conflicts with authority.

PENDING24

URGENT08

Advanced Filters

CASE ID	ISSUE TYPE	CLAIMANT	GARAGE	PRIORITY	STATUS
#CP-990234	Overcharging Dispute	Arthur Dent	Metro Engine Works	HIGH	Under Review
#CP-88120	Poor Quality of Work	Tricia McMillan	QuickFix Autos	MEDIUM	New
#CP-87554	Rude Behavior	Ford Prefect	The Starship Repair	LOW	Resolved

EVIDENCE DOSSIER

CASE CORRESPONDENCE

JUDICIAL RULING

GarageGo

Admin Portal

Dashboard

Analytics

Users

Garages

Bookings

Emergency

Payments

Reviews

Complaints

Reports

Settings

Complaints & Disputes

Mediating and resolving marketplace conflicts with authority.

PENDING24

URGENT08

Advanced Filters

CASE ID	ISSUE TYPE	CLAIMANT	GARAGE	PRIORITY	STATUS
#CP-990234	Overcharging Dispute	Arthur Dent	Metro Engine Works	HIGH	Under Review
#CP-88120	Poor Quality of Work	Tricia McMillan	QuickFix Autos	MEDIUM	New
#CP-87554	Rude Behavior	Ford Prefect	The Starship Repair	LOW	Resolved

EVIDENCE DOSSIER

CASE CORRESPONDENCE

JUDICIAL RULING

GarageGo

Admin Portal

Secure access for platform administrators

Email Address

admin@garagego.com

Password

Forgot password?

Remember me for 30 days

Sign In →

Empowering Sri Lanka's Automotive Ecosystem

Real-time oversight of garage operations, booking analytics, and emergency service dispatch across the island.

1.2k+

Active Garages

4.8/5

User Rating

24/7

Emergency Support

36

6.3 Navigation Flow

The navigation flow enforces role-based routing at the React Navigation level using JWT role claims. Unauthenticated users are redirected to the Login screen. An invalid or expired token triggers automatic redirect to Login with a toast notification.

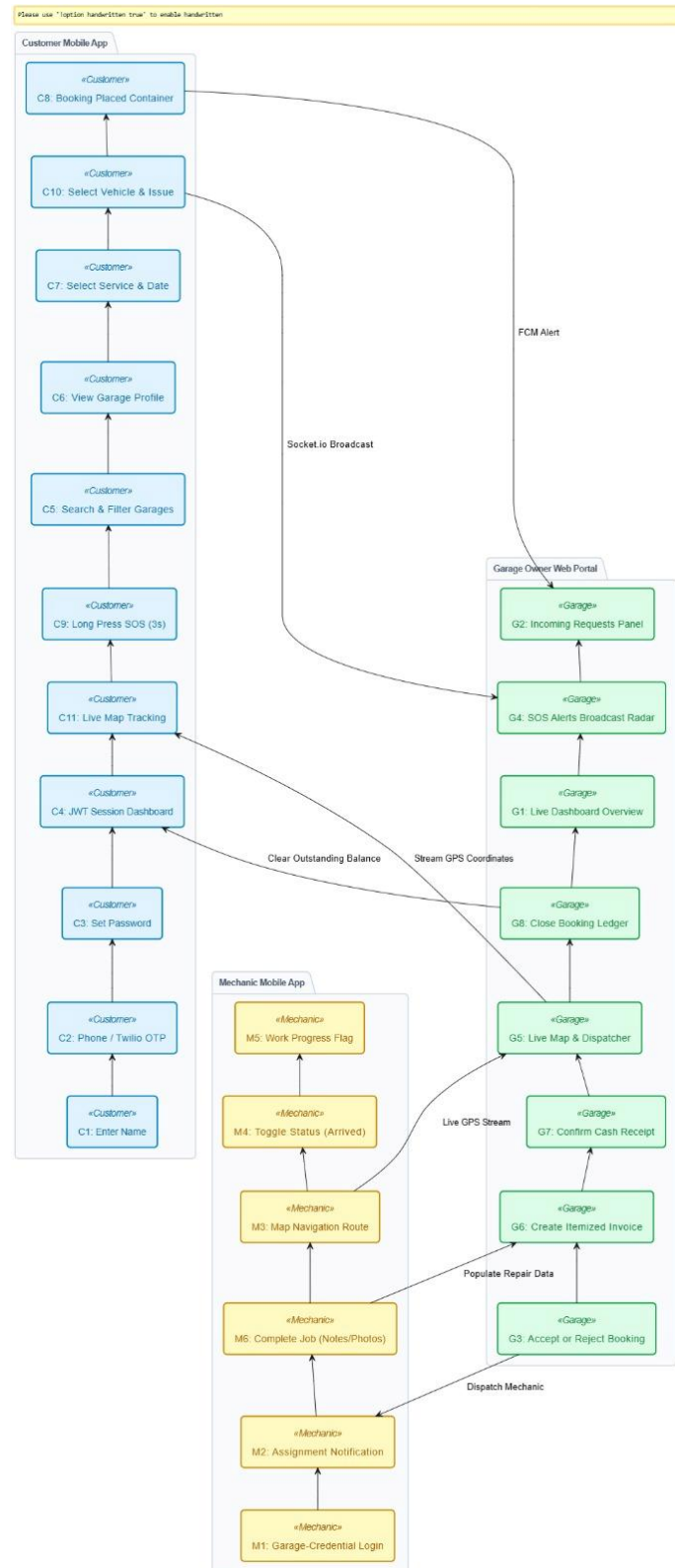


Figure 11: GarageGo – Navigation Flow Diagram [see documents/navigation-flow-diagram.png]

An unauthenticated user lands on the Splash/Login screen and may proceed to Registration or Login. Upon successful login as CUSTOMER, the user is directed to the Home Dashboard, from which Garage Search, Booking Creation, Emergency SOS, Mechanic Tracking, Service History, and Profile screens are accessible. Upon successful login as GARAGE_OWNER, the user is directed to the Garage Dashboard, from which Booking Requests, Emergency Alerts, Mechanic Assignment, Service Management, and Analytics screens are accessible. Upon login as MECHANIC, the user is directed to the Active Jobs screen with GPS navigation controls. Upon login as ADMIN, the browser-based Admin Dashboard provides Garage Approval, User Management, Complaint Resolution, and Platform Analytics screens. Logout from any screen returns to the Login screen.

Chapter 7: Security Design

The GarageGo system implements a defense-in-depth security strategy with independent protection layers: Transport Security (TLS 1.2+/WSS), Authentication (JWT + OTP), Authorisation (RBAC), Data Security (BCrypt + AES-256), and Input Validation (server-side + schema validation). A breach of one layer does not compromise the entire system.

7.1 Authentication and Authorisation Strategy

Authentication is implemented using JSON Web Tokens (JWT) issued by the Node.js/Express API upon successful credential verification plus OTP confirmation. The access token has a validity period of 15 minutes (NFR-S1). A refresh token valid for 7 days is stored in an HttpOnly cookie to prevent JavaScript access and is used to obtain new access tokens transparently.

- All users (Customer, Garage Owner, Mechanic) verify their phone number via Twilio OTP during registration (FR-01; BR-01). Garage Owner email verification is also mandatory (BR-02).
- JWT tokens carry the role claim (CUSTOMER | GARAGE_OWNER | MECHANIC | ADMIN) used for RBAC enforcement at the Express middleware layer.
- All API endpoints except /auth/register and /auth/login require a valid JWT. Missing or expired tokens return HTTP 401 (NFR-S1).
- Admin role access is enforced via middleware verifying `role === 'ADMIN'` before any /admin/* route is processed.

7.2 Security Design Summary

Table 23: Security Design Summary

Security Aspect	Mechanism	Details
Authentication	JWT (HS256) + Twilio OTP	Access tokens expire after 15 minutes (NFR-S1); refresh tokens stored in HttpOnly cookie (7-day expiry)
Authorisation	RBAC via Express middleware	Roles: CUSTOMER, GARAGE_OWNER, MECHANIC, ADMIN. Middleware enforces role per route group.
Password Storage	BCrypt (cost factor 12)	Passwords never stored in plain text; BCrypt applied on registration and password reset (NFR-S2)
Data in Transit	TLS 1.2+ (HTTPS enforced) + WSS	All REST endpoints served over HTTPS; Socket.io connections use WSS protocol (NFR-S3)
Data at Rest	AES-256 for NIC; BCrypt for passwords	NIC numbers encrypted before database storage; bcrypt for all passwords (NFR-S2)
Input Validation	Server-side Express-validator + Joi schema	All API request bodies validated against Joi schemas; length limits enforced; reject unknown fields
SQL Injection	Parameterised queries via pg/Sequelize	No raw SQL string concatenation; all queries use prepared statements

Security Aspect	Mechanism	Details
XSS Prevention	React Native + React.js auto-escaping	Framework escapes all dynamic content; Content-Security-Policy headers on Admin Dashboard
Race Conditions	SELECT FOR UPDATE (PostgreSQL advisory lock)	Prevents duplicate emergency assignments when multiple garages accept simultaneously (FR-06)
File Upload	Extension + MIME type + size validation	Garage photos: JPEG/PNG only, max 10 MB. Documents: PDF only, max 5 MB. Validated server-side.
Audit Logging	Morgan HTTP logger + custom event logger	All authentication events, admin actions, and emergency assignments logged with timestamp, IP, and userId

7.3 Input Validation and Sanitisation Strategy

All input data is validated at two independent levels. At the client side, React Native form validation enforces field-level rules including required fields, phone format (E.164), email format, NIC pattern, and numeric ranges for GPS coordinates. At the server side, Joi schema validation applied to all request bodies enforces constraints declaratively. Input exceeding defined length limits is rejected with HTTP 400 Bad Request. All string inputs are trimmed of leading and trailing whitespace before processing. Review text is scanned for a configurable list of offensive patterns before storage (UC-09 Alt Flow 2). File uploads are restricted to approved MIME types with server-side size enforcement.

7.4 Encryption Standards

Table 24: Encryption Standards

Data Category	Standard	Implementation
Data in Transit (REST)	TLS 1.2+	Enforced at reverse proxy (Nginx/load balancer); HTTP permanently redirected to HTTPS
Data in Transit (WebSocket)	WSS (TLS 1.2+)	Socket.io configured with HTTPS server; plaintext WS connections rejected
Passwords at Rest	BCrypt (cost factor 12)	Applied in Express auth middleware before any database write (NFR-S2)
NIC Numbers at Rest	AES-256-GCM	Application-layer encryption using a server-side key; encrypted value stored in DB
JWT Signature	HS256 (HMAC-SHA256)	Signing secret stored in environment variable; never embedded in application code
File Storage	Provider-managed encryption	Cloudinary or S3 server-side encryption applied automatically to all uploaded files

7.5 OWASP Top 10 Considerations

Table 25: OWASP Top 10 – GarageGo Controls

#	OWASP Risk	GarageGo Control
A01	Broken Access Control	RBAC middleware enforces role-based route access on every request; JWT validated on all protected endpoints (NFR-S1)
A02	Cryptographic Failures	TLS 1.2+ for all transit; BCrypt cost 12 for passwords; AES-256-GCM for NIC data; no sensitive data in JWT payload
A03	Injection	Parameterised queries via pg/Sequelize for all DB operations; Joi schema validation rejects unexpected fields
A04	Insecure Design	Threat modelling conducted during sprint; SELECT FOR UPDATE lock prevents race conditions in emergency assignment
A05	Security Misconfiguration	Environment variables for all secrets; /api/docs accessible in development mode only (NFR-M3); CORS restricted to known origins
A06	Vulnerable Components	npm audit run in CI/CD pipeline; dependencies pinned in package-lock.json; critical vulnerabilities block pipeline
A07	Authentication Failures	OTP required for all new accounts (BR-01); JWT 15-minute expiry (NFR-S1); no brute-force protection placeholder (post-MVP: rate limiting)
A08	Data Integrity Failures	JWT signature verification on every request; file upload MIME type and size validation; booking state machine enforces valid transitions only
A09	Logging Failures	All authentication events, admin actions, and emergency assignments logged with timestamp, IP, and userId via Morgan
A10	SSRF	No server-side URL fetching from user input; Google Maps API calls use server-side SDK with fixed endpoints only

Chapter 8: Interface Design

This chapter specifies all external interfaces (third-party APIs and services) and internal module interfaces for the GarageGo platform.

8.1 External Interfaces

Table 26: External Interface Specifications

Interface	Provider	Purpose	Key Endpoints / Integration
Google Maps Geocoding API	Google Maps Platform	Convert garage addresses to lat/lng coordinates during registration (FR-10)	GET /geocode/json?address=&key= Returns lat, lng, formatted_address
Google Maps Distance Matrix API	Google Maps Platform	Calculate distance and ETA from mechanic to customer location during emergency (FR-07)	GET /distancematrix/json?origins=&destinations=&key= Returns distance_value (m), duration_value (s)
Google Maps Directions API	Google Maps Platform	Generate turn-by-turn navigation for mechanic app (FR-15)	GET /directions/json?origin=&destination=&key= Returns encoded polyline for map display
Twilio SMS API	Twilio Inc.	Send OTP codes for phone verification (FR-01; BR-01)	POST /2010-04-01/Accounts/{SID}/Messages Body: { To, From, Body (6-digit OTP) } Response: messageSid
Firebase Cloud Messaging	Google Firebase	Deliver push notifications to Customer, Garage Owner, and Mechanic apps (FR-14)	POST https://fcm.googleapis.com/fcm/send Body: { token, notification: { title, body }, data: { type, payload } }
Cloudinary / Amazon S3	Cloudinary / AWS	Store garage photos, vehicle images, and complaint evidence (SRS §1.2.1)	Cloudinary Upload API: POST /auto/upload Returns secure_url for storage in DB

8.2 Internal Module Interfaces

Table 27: Internal Module Interface Specifications

From Module	To Module	Interface Type	Key Interactions
Express REST API	PostgreSQL 15	TCP/IP via node-postgres (pg)	Parameterised SQL queries via pg.Pool; connection pool max 20 per instance; all queries use prepared statements

From Module	To Module	Interface Type	Key Interactions
Express REST API	Socket.io Server	In-process EventEmitter / same Node.js process	EmergencyService calls <code>io.to(garageRoom).emit('emergency_alert', payload)</code> ; GarageService calls <code>io.to(customerRoom).emit('booking_status', payload)</code>
Express REST API	Redis 7	TCP/IP via ioredis client	OTP storage: <code>SET otp:{customerId}{code}</code> EX 300; Socket.io session adapter: <code>socket.io-redis</code> ; garage list cache: <code>GET/SET garages:nearby:{geohash}</code>
Socket.io Server	Redis 7 (Pub/Sub)	Redis Pub/Sub via <code>socket.io-redis</code> adapter	Enables Socket.io message broadcasting across multiple horizontally scaled API instances (NFR-SC1)
Express REST API	NotificationService	In-process function call	<code>NotificationService.sendPush(token, type, payload)</code> wraps FCM HTTP API call; <code>NotificationService.sendSMS(phone, message)</code> wraps Twilio API call
Express REST API	Google Maps Client	HTTPS via <code>@googlemaps/google-maps-services-js</code>	<code>GarageService.geocodeAddress(address)</code> ; <code>EmergencyService.findNearbyGarages(lat, lng, radius)</code> ; <code>EmergencyService.getETA(origin, destination)</code>
Customer App / Garage App / Mechanic App	Express REST API	HTTPS/REST (JSON)	All CRUD operations, authentication, and booking management via versioned REST endpoints (<code>/api/v1/*</code>)
Mechanic App	Socket.io Server	WSS WebSocket	Mechanic emits <code>location_update</code> events every 5s; receives <code>job_assigned</code> and <code>navigation_update</code> events
Customer App	Socket.io Server	WSS WebSocket	Customer receives <code>mechanic_location</code> , <code>booking_status</code> , and <code>emergency_assigned</code> events in real time

Chapter 9: Deployment Design

This chapter describes the hosting plan, environment specifications, deployment network architecture, and CI/CD pipeline for the GarageGo platform. All components are containerised using Docker for environment parity between development, staging, and production.

9.1 Hosting Plan

Table 28: GarageGo Deployment Components

Component	Technology	Specification
Customer Mobile App	React Native (Expo)	Published to Apple App Store and Google Play Store; OTA updates via Expo EAS
Garage Owner Mobile App	React Native (Expo)	Published to App Store and Play Store alongside Customer App
Mechanic Mobile App	React Native (Expo)	Published to App Store and Play Store; GPS always-on during active dispatch
Admin Web Dashboard	React.js (Vite)	Served as static build from Nginx container; HTTPS with valid TLS certificate
REST API Server	Node.js 20 LTS / Express	Docker container behind load balancer; min 1 instance, scales to 4 (NFR-SC3)
Socket.io Server	Node.js 20 LTS / Socket.io 4	Co-located with REST API in same Node.js process; Redis adapter for multi-instance pub/sub (NFR-SC1)
Database	PostgreSQL 15	Managed cloud service; daily automated backups retained 30 days (NFR-R3); monthly partitioned bookings table (NFR-SC2)
Cache / Session	Redis 7	Managed cloud service; used for OTP TTL, Socket.io sessions, and garage list cache
File Storage	Cloudinary or Amazon S3	Garage photos, vehicle images, invoice PDFs, complaint evidence (SRS §1.2.1)
Reverse Proxy	Nginx	Routes /api/* to Node.js; serves Admin Dashboard static files; enforces HTTPS redirect

9.2 Environment Specification

Table 29: Environment Specification

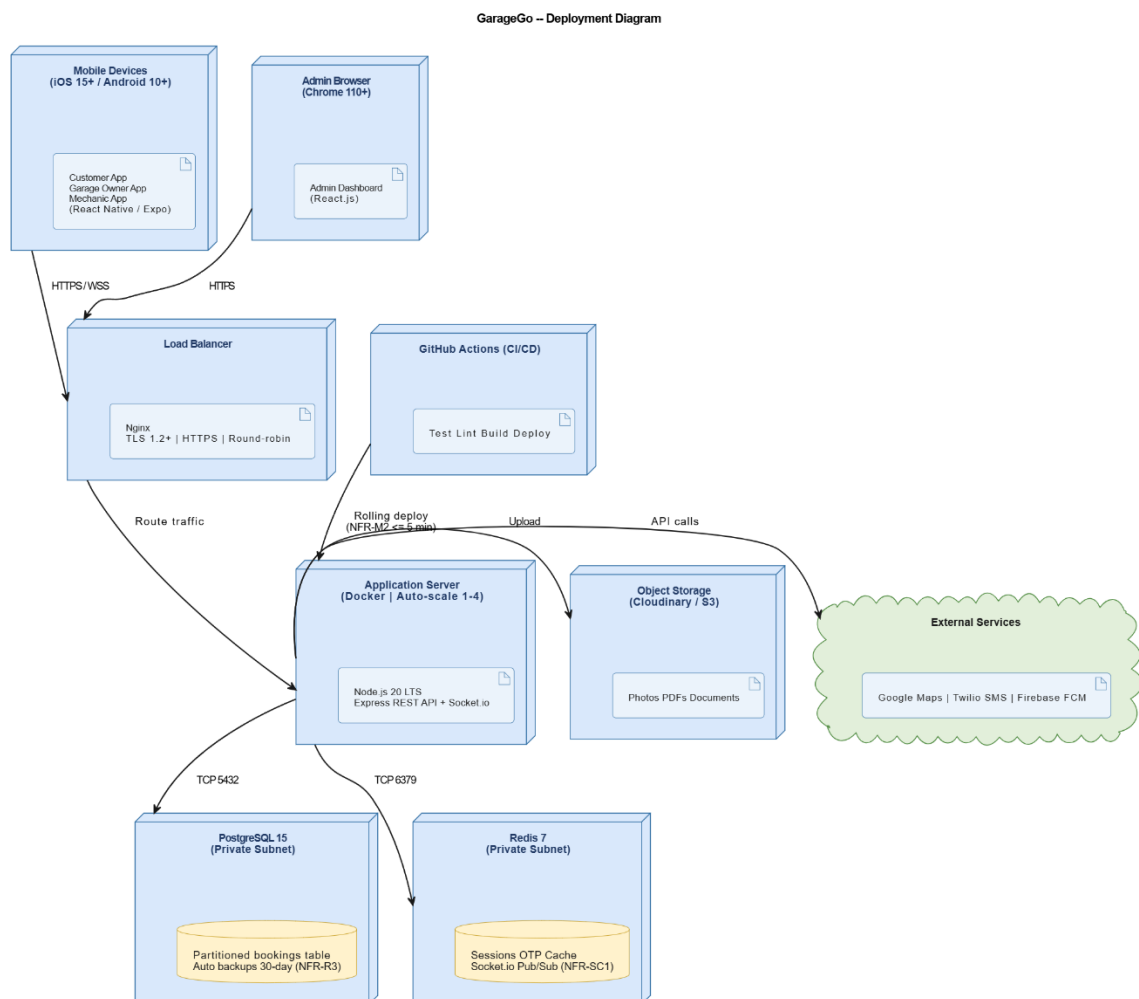
Attribute	Development	Staging	Production
Infrastructure	Local Docker Compose	Single cloud VM	Multiple Docker containers behind load balancer
Database	Local PostgreSQL 15	Managed PostgreSQL (single instance)	Managed PostgreSQL (with automated backups)
Redis	Local Redis 7 container	Managed Redis (single node)	Managed Redis (single node, persistent)

Attribute	Development	Staging	Production
Branch	feature/* branches	develop branch	main branch
Logging	Full stack traces (DEBUG)	INFO-level logging	ERROR level + audit events only
HTTPS	Self-signed certificate	Valid TLS certificate (staging domain)	Valid TLS certificate (production domain)
External APIs	Mock/test mode	Twilio test credentials; FCM sandbox	Live production API keys
Unit Test Coverage	≥70% enforced (NFR-M1)	≥70% enforced	≥70% enforced via CI gate

9.3 Network Architecture

The network architecture places all backend resources within a private network. The load balancer is the only publicly reachable entry point for external traffic. The PostgreSQL and Redis instances reside in a private subnet with no direct internet access. All outbound calls from the API server (to Google Maps, Twilio, FCM) are made via NAT.

Figure 12: GarageGo – Deployment Network Diagram [see documents/diagrams/deployment-diagram.png]



Chapter 10: Non-Functional Design Decisions

This chapter documents how each of the fourteen non-functional requirements defined in the GarageGo SRS v0.1 (Sections 5.2–5.7) is addressed by the system design. Each entry states the NFR, the design decision that satisfies it, and the responsible component or configuration.

10.1 Performance

Table 30: Performance NFR Design Decisions

NFR ID	Requirement (SRS §5.2)	Design Decision	Responsible Component
NFR-P1	95% of REST API requests respond within 2 seconds under 50 concurrent users	Stateless Express API with connection pooling (pg.Pool max 20); Redis caching for garage list results; indexed PostgreSQL queries on garageId, customerId, status	Express API, pg.Pool, Redis, PostgreSQL indices
NFR-P2	Emergency SOS broadcast delivered to all eligible garages within 2 seconds of SOS trigger	Socket.io emits directly to garage-specific rooms after EmergencyService.findNearbyGarages() resolves; no polling; Redis pub/sub ensures cross-instance delivery	Socket.io Server, EmergencyService, Redis pub/sub
NFR-P3	Mechanic GPS relay to customer within 1 second; maximum 6-second end-to-end interval	Mechanic app emits location_update every 5 seconds; Socket.io forwards immediately to customer room; no intermediate persistence delay in the critical path	Socket.io Server, Mechanic App, Customer App

10.2 Security

Table 31: Security NFR Design Decisions

NFR ID	Requirement (SRS §5.3)	Design Decision	Responsible Component
NFR-S1	All protected endpoints require valid JWT; 15-minute access token expiry; 7-day refresh token expiry	Express auth middleware validates JWT on every request to /api/v1/* except /auth/register and /auth/login; HS256 signing; refresh token in HttpOnly cookie	Express JWT Middleware, AuthService
NFR-S2	Passwords hashed with BCrypt cost factor ≥ 12 ; no plain-text storage or logging	bcryptjs library with saltRounds=12 applied in AuthService.hashPassword() before any DB write; no password field in any API response or log output	AuthService, bcryptjs
NFR-S3	All communication over TLS 1.2+; WebSocket over WSS; HTTP redirected to HTTPS	Nginx reverse proxy configured with TLS 1.2 minimum; HTTP permanently redirected to HTTPS (301);	Nginx, Socket.io Server

NFR ID	Requirement (SRS §5.3)	Design Decision	Responsible Component
		Socket.io server instantiated with HTTPS server object	

10.3 Usability

Table 32: Usability NFR Design Decisions

NFR ID	Requirement (SRS §5.4)	Design Decision	Responsible Component
NFR-U1	SOS button visible without scrolling; minimum 48×48 dp tap target	SOS button implemented as full-width TouchableOpacity with height=80 and marginBottom=24 on the Home Dashboard, above the fold on all supported screen sizes; minWidth and minHeight set to 48 in StyleSheet	Customer App – HomeScreen.jsx
NFR-U2	Full registration completable within 3 minutes on 4G	Registration form uses a single-page scrollable layout rather than multi-step wizard; Twilio OTP auto-detected by iOS/Android SMS listener; all fields use appropriate keyboard types (numeric, email-address) to reduce input friction	Customer App – RegisterScreen.jsx
NFR-U3	User-friendly error message displayed within 3 seconds of connectivity loss; no raw error codes shown	NetInfo event listener detects connectivity change and dispatches a Redux action; ErrorBanner component renders a human-readable message within 1 render cycle; all API errors mapped to user-friendly strings in errorMessages.js	Customer App – NetworkMonitor.js, ErrorBanner.jsx

10.4 Reliability

Table 33: Reliability NFR Design Decisions

NFR ID	Requirement (SRS §5.5)	Design Decision	Responsible Component
NFR-R1	API server uptime ≥99.5% during peak hours (7AM–9PM SLST)	Docker container restarts automatically on crash (restart: always); health check endpoint	Docker, Load Balancer, Health Check Endpoint

NFR ID	Requirement (SRS §5.5)	Design Decision	Responsible Component
		/health monitored by load balancer; unhealthy instances replaced automatically	
NFR-R2	Webhook failure recovery: reconciliation polling within 10 minutes	A Bull queue job (ReconciliationJob) runs every 10 minutes checking for bookings or notifications in PENDING state older than 10 minutes without a webhook confirmation; re-triggers notification delivery	Bull Queue (Redis-backed), ReconciliationJob
NFR-R3	PostgreSQL backed up every 24 hours; 30-day retention; recovery within 2 hours	Managed PostgreSQL provider automated daily snapshots with 30-day retention policy configured; backup restore procedure documented and tested in operations runbook	Managed PostgreSQL, Operations Runbook

10.5 Scalability

Table 34: Scalability NFR Design Decisions

NFR ID	Requirement (SRS §5.6)	Design Decision	Responsible Component
NFR-SC1	Socket.io server supports ≥ 500 concurrent connections; message delivery ≤ 1 second at full load	Socket.io configured with socket.io-redis adapter (ioredis); Redis pub/sub channel fans out events across all horizontally scaled API instances; each instance can handle ~ 200 connections; 3 instances = 600 capacity	Socket.io Server, Redis pub/sub, socket.io-redis
NFR-SC2	PostgreSQL bookings table supports monthly horizontal partitioning; queries ≤ 500 ms at 100,000+ records	bookings table declared with PARTITION BY RANGE (scheduledAt); monthly child partitions created by a scheduled pg-cron job; partial index on status per partition; queries specify scheduledAt to benefit from partition pruning	PostgreSQL 15, pg-cron, bookings partitioning
NFR-SC3	Adding second API instance increases throughput by $\geq 80\%$	Stateless Express API (no in-process session state; JWT-based auth); load balancer distributes requests round-robin; Redis handles all shared state; see deployment diagram (Figure 12)	Express API, Load Balancer, Redis

10.6 Maintainability

Table 35: Maintainability NFR Design Decisions

NFR ID	Requirement (SRS §5.7)	Design Decision	Responsible Component
NFR-M1	Backend unit test coverage $\geq 70\%$ via Jest; report generated on every PR	Jest configured with <code>--coverage</code> flag in npm test script; coverage thresholds set in <code>jest.config.js</code> (lines: 70, functions: 70, branches: 70); GitHub Actions workflow runs tests on every PR and posts coverage report as PR comment; PR merge blocked if threshold not met	Jest, GitHub Actions, <code>jest.config.js</code>
NFR-M2	Each component independently redeployable via Docker; redeployment ≤ 5 minutes; zero downtime for other components	API server, Admin Dashboard, and Nginx each packaged as separate Docker images; Docker Compose or Kubernetes rolling update deploys new image while old container continues serving requests; health check confirms readiness before traffic switch	Docker, Rolling Update Strategy
NFR-M3	All public API endpoints documented in OpenAPI 3.0; <code>/api/docs</code> accessible in development mode	<code>swagger-jsdoc</code> generates OpenAPI 3.0 spec from JSDoc annotations on route handlers; <code>swagger-ui-express</code> serves interactive documentation at <code>/api/docs</code> when <code>NODE_ENV=development</code> ; spec file committed to <code>documents/api-spec/openapi.yaml</code>	<code>swagger-jsdoc</code> , <code>swagger-ui-express</code> , OpenAPI 3.0

References

Table 36: References

#]	Reference
[1]	IEEE, "IEEE Std 1016-2009: IEEE Standard for Information Technology – Systems Design – Software Design Descriptions," IEEE, 2009.
[2]	IEEE, "IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications," IEEE, 1998.
[3]	ISO/IEC, "ISO/IEC 25010: System and Software Quality Models," ISO/IEC, 2011.
[4]	R. S. Pressman, Software Engineering: A Practitioner's Approach, 8th ed. New York: McGraw-Hill, 2014.
[5]	I. Sommerville, Software Engineering, 10th ed. Harlow: Pearson, 2016.
[6]	Node.js Foundation, "Node.js 20 LTS Documentation," [Online]. Available: https://nodejs.org/en/docs/ . [Accessed: Jun. 2026].
[7]	Socket.io, "Socket.IO Documentation," [Online]. Available: https://socket.io/docs/v4/ . [Accessed: Jun. 2026].
[8]	Expo, "Expo Documentation," [Online]. Available: https://docs.expo.dev/ . [Accessed: Jun. 2026].
[9]	PostgreSQL Global Development Group, "PostgreSQL 15 Documentation," [Online]. Available: https://www.postgresql.org/docs/15/ . [Accessed: Jun. 2026].
[10]	Redis Ltd., "Redis 7 Documentation," [Online]. Available: https://redis.io/docs/ . [Accessed: Jun. 2026].
[11]	Google, "Google Maps Platform Documentation," [Online]. Available: https://developers.google.com/maps/documentation . [Accessed: Jun. 2026].
[12]	Twilio, "Twilio Messaging API Reference," [Online]. Available: https://www.twilio.com/docs/messaging . [Accessed: Jun. 2026].
[13]	Firebase, "Firebase Cloud Messaging Documentation," [Online]. Available: https://firebase.google.com/docs/cloud-messaging . [Accessed: Jun. 2026].
[14]	OWASP, "OWASP Top Ten," [Online]. Available: https://owasp.org/www-project-top-ten/ . [Accessed: Jun. 2026].
[15]	W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," [Online]. Available: https://www.w3.org/TR/WCAG21/ . [Accessed: Jun. 2026].
[16]	GarageGo Team, "GarageGo Software Requirements Specification v0.1," University of Kelaniya, May 2026.
[17]	University of Kelaniya, "SENG 31242 System Design Project – Course Guideline & Industrial Standards v1.0," Software Engineering Teaching Unit, 2026.